Escuela
Politécnica
Superior

Integración de Blockchain para la trazabilidad en el entrenamiento distribuido de modelos de IA



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:
David Piotr Znojek
Tutor/es:
Francisco A. Pujol López



Universitat d'Alacant
Universidad de Alicante

Julio 2026

Integración de Blockchain para la trazabilidad en el entrenamiento distribuido de modelos de IA

Autor
David Piotr Znojek

Tutor/es
Francisco A. Pujol López
Tecnología Informática y Computación



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Julio 2026

Resumen

Este Trabajo de Fin de Grado tiene como objetivo validar, investigar y evaluar la viabilidad de integrar la tecnología Blockchain como capa de auditoría y trazabilidad durante el proceso de entrenamiento de un modelo de Inteligencia Artificial descentralizado. Se presenta un desarrollo de la problemática del mundo actual en cuanto a los modelos de Inteligencia Artificial, las posibles fallas de seguridad que se pueden producir durante el entrenamiento de estos modelos y, sobre todo, se proponen soluciones para aumentar la capacidad de auditoría e integridad de los datos mediante un sistema basado en trazabilidad y verificación.

En una red de entrenamiento de un modelo de Inteligencia Artificial distribuido, existe un peligro importante de manipulación de datos por medio de inyección de datos erróneos o de procedencia desconocida, los cuales pueden alterar los resultados finales del entrenamiento. Estos modelos puestos en sistemas en producción con los cuales interactúan personas en el mundo real, podrían dar lugar a acontecimientos fatales en cuanto a la integridad física de las personas. Es por ello que es estrictamente necesario utilizar un sistema que proporcione una confianza y seguridad mediante trazabilidad y auditoría, permitiendo detectar a tiempo incoherencias en los registros utilizados durante los entrenamientos. Es allí donde entra la Blockchain.

Para llevar a cabo la experimentación y para la generación de resultados, el trabajo ha sido enfocado en el ámbito de los vehículos autónomos, los cuales utilizan modelos entrenados de Inteligencia Artificial para detectar diferentes situaciones en la carretera, tanto favorables como adversas. Se intentará dar una visión clara y elaborada de las ventajas y de los inconvenientes que implica desarrollar una red de Blockchain que se ocupe de garantizar la seguridad de los datos que fluyen durante el entrenamiento de los modelos de Inteligencia Artificial. Además, se analiza la sostenibilidad de la red con diferentes métricas que muestran el gasto real, tanto de tiempo como de recursos.

Agradecimientos

A mi familia, por su apoyo sin límites a lo largo de todos estos años y por creer en mí siempre. Nada de esto habría sido posible sin vosotros.

A mis amigos, por estar siempre ahí, por las risas en los momentos buenos y por el apoyo en los difíciles. Gracias por hacer este camino mucho más llevadero.

A mi tutor, por su orientación y sus consejos a lo largo de todo el desarrollo de este trabajo. Gracias por guiarme y por confiar en este proyecto desde el principio.

Y, por último, quiero agradecerme también a mí mismo por la constancia, por no rendirme cuando las cosas se complican y por seguir siempre adelante hasta el final.

Este trabajo es también fruto de ese esfuerzo.

A mis padres Beata y Piotr, a mis hermanos Ana y Pedro y a los alumnos VIP del grupo ARA de la promoción 2022-2026, gracias por vuestra ayuda.

Índice general

1 Introducción **1**
1.1 Problemática . . . 1
1.2 Solución propuesta . . . 1
1.3 Dilema de seguridad y rendimiento . . . 2
1.4 Motivación del proyecto: Validación de una arquitectura Blockchain-IA híbrida 2

2 Marco Teórico **5**
2.1 Blockchain . . . 5
2.1.1 Concepto de Blockchain . . . 5
2.1.2 Funciones hash criptográficas . . . 6
2.1.3 Tipos de redes Blockchain . . . 7
2.1.4 Elementos de la red . . . 8
2.1.4.1 Nodo . . . 8
2.1.4.2 Red y Peers . . . 8
2.1.4.3 Consenso . . . 9
2.1.4.4 IBFT 2.0 . . . 10
2.1.4.5 Validadores . . . 11
2.1.4.6 Gas . . . 12
2.1.5 Máquina Virtual de Ethereum (EVM) . . . 13
2.1.6 Smart Contracts . . . 13
2.1.7 Plataformas empresariales: GoQuorum y Hyperledger Besu . . . 14
2.1.8 Privacidad con Tessera . . . 15
2.1.9 Trazabilidad con Blockchain . . . 16
2.2 Almacenamiento distribuido: IPFS . . . 18
2.2.1 Concepto de IPFS . . . 18
2.2.2 Direccionamiento por contenido (CID) . . . 18
2.3 Inteligencia Artificial . . . 20
2.3.1 Concepto de Inteligencia Artificial . . . 20
2.3.2 Machine Learning . . . 20
2.3.3 Aprendizaje Federado . . . 20
2.3.4 Algoritmo de agregación: FedAvg . . . 21
2.3.5 Framework de Flower . . . 22
2.3.6 Dataset utilizado . . . 22
2.4 Integración de Inteligencia Artificial con Blockchain . . . 23
2.4.1 Problemas del aprendizaje federado . . . 23
2.4.2 Uso de Blockchain como solución . . . 23

3 Objetivos **25**
3.1 Objetivo general . . . 25

3.2 Objetivos específicos 25

4 Metodología **27**
4.1 Enfoque de trabajo 27
4.2 Fases del proyecto 27
4.3 Stack tecnológico 28
4.3.1 Blockchain 28
4.3.2 Aprendizaje federado e Inteligencia Artificial 28
4.3.3 Almacenamiento e integración 29
4.3.4 Lenguaje y entorno 29
4.4 Estructura del proyecto 29

5 Desarrollo **31**
5.1 Red de Blockchain 31
5.1.1 Arquitectura: ConsenSys y Hyperledger Besu 31
5.1.2 Configuración de los Nodos 32
5.1.3 Desarrollo del Smart Contract: FLRegistry 33
5.1.4 Despliegue de la red Blockchain 37
5.2 Red de entrenamiento distribuido 41
5.2.1 Arquitectura 41
5.2.2 Modelo de IA usado 41
5.2.3 Proceso de entrenamiento federado 42
5.2.4 Comunicación cliente-servidor 44
5.3 Integración IA + Blockchain 45
5.3.1 Conexión de ambos sistemas 45
5.3.2 Flujo completo: preparación, entrenamiento, agregación y registro . . 45
5.3.2.1 Preparación inicial (una sola vez) 46
5.3.2.2 Durante cada ronda 47
5.3.3 Trazabilidad y garantías 49
5.4 Métricas recogidas 51
5.4.1 Métricas del cliente 51
5.4.2 Métricas del servidor 51
5.4.3 Métricas sin Blockchain 52
5.4.4 Justificación de las métricas elegidas 52

6 Resultados **53**
6.1 Entorno de pruebas 53
6.2 Diseño experimental 54
6.2.1 Despliegue del entorno 54
6.2.2 Comparaciones planteadas 55
6.2.3 Aspectos adicionales de la ejecución 55
6.3 Resultados 56
6.3.1 Convergencia del modelo 56
6.3.2 Visión general de los tiempos 57
6.3.3 Coste de la integración con Blockchain 59
6.3.4 Efecto de la distribución en varias máquinas 60

6.3.5 Impacto del número de validadores 61
6.3.6 Consumo de gas 62
6.3.7 Discusión global 63

7 Conclusiones **65**
7.1 Recapitulación del trabajo 65
7.2 Revisión de los objetivos 65
7.3 Conclusiones principales 66
7.4 Limitaciones 67
7.5 Trabajo futuro 67

Bibliografía **69**

Índice de figuras

1.1 Arquitectura a validar 3

2.1 Encadenamiento de bloques en red Blockchain 6
2.2 Comparativa de almacenamiento de archivos 19
2.3 Flujo del aprendizaje federado 21
2.4 Arquitectura del sistema completa 24

5.1 Conexión establecida con la red Blockchain desplegada 38
5.2 Interfaz web Quorum Explorer 38
5.3 Despliegue del contrato en la red Blockchain 40
5.4 Ejemplo de ejecución del entrenamiento de un cliente 43
5.5 Final del entrenamiento 44
5.6 Preparación inicial del sistema 46
5.7 Inicio de ronda y verificación de los pesos iniciales por parte del cliente . 47
5.8 Entrenamiento y subida de pesos del cliente 48
5.9 Verificación por parte del servidor y fin de ronda 49

6.1 Evolución de la pérdida de validación a lo largo de las 30 rondas. 56
6.2 Evolución del tiempo total por ronda en las cinco configuraciones. 58
6.3 Coste temporal de la integración con Blockchain, en local y en distribuido. 59
6.4 Efecto de distribuir el sistema en dos máquinas. 60
6.5 Tiempo medio de confirmación de transacción en las configuraciones con Blockchain. 61
6.6 Consumo de gas por transacción a lo largo de las rondas. 62

Índice de tablas

2.1 Comparativa de los tipos de redes Blockchain. 7

6.1 Características de la Máquina 1 53
6.2 Características de la Máquina 2 53
6.3 Configuraciones experimentales 54
6.4 Tiempo total por ronda en cada configuración. 57

Índice de Códigos

5.1 Contrato FLRegistry 34
5.2 Generación de los artefactos del quickstart 37
5.3 Arranque de la red Besu 37
5.4 Verificación de la sincronización 37
5.5 Script de despliegue del contrato (fragmentado) 39

1 Introducción

El mundo actual está en auge en el desarrollo de modelos de Inteligencia Artificial, donde el uso masivo de datos personales, como lo son la localización, las tendencias del consumo o los intereses, se han vuelto esenciales para la evolución de la tecnología. Es por ello que garantizar la integridad de los datos de los usuarios ya no es una opción, sino que es una necesidad, tanto técnica como ética.

Combinar la potencia y el conocimiento que nos permite obtener la Inteligencia Artificial, con la capacidad de seguridad y auditoría que permite Blockchain, puede parecer un desafío difícil e importante, pero es posible.

Por otro lado, el proceso de entrenamiento de un modelo centralizado puede llegar a ser realmente extenso, por lo que dejar todo el trabajo a una sola máquina en algunas ocasiones, si la cantidad de datos es enorme y ésta no tiene la capacidad suficiente para soportar el entrenamiento, puede ser inviable. Es por ello que delegar el trabajo a diversas máquinas, como es en el caso del entrenamiento distribuido, y agregar posteriormente los resultados es una medida que acelera el entrenamiento y reduce el consumo de energía de cada una de manera notable.

1.1 Problemática

Los modelos tradicionales son centralizados, es decir, las empresas o usuarios que realizan el entrenamiento controlan todos los datos y conocen su procedencia. En el caso del entrenamiento distribuido, existe el riesgo de conexión de usuarios maliciosos que pretendan alterar los resultados por diversos motivos. Por ejemplo, un usuario no autorizado entrena un modelo que no detecte u omita adrede las señales de STOP en la carretera. Si este usuario se conecta a la red de entrenamiento y esta no tiene las medidas de seguridad necesarias de integridad y trazabilidad, el usuario podrá introducir la información que quiera y la combinará con el resto. ¿Qué ocurre si este usuario repite este proceso las veces suficientes como para que sus contribuciones supongan un peso importante en el resultado final? Los vehículos de la red tenderán a omitir las señales de STOP y provocarán finalmente un accidente. Es por ello que mantener un registro inmutable de las contribuciones de cada uno de los miembros de la red es absolutamente esencial para detectar incidencias y poder evitar desgracias a tiempo.

1.2 Solución propuesta

Para aumentar esta seguridad, la cual es esencial en este tipo de entrenamientos, se ha propuesto el uso de una tecnología que sirve de libro de contabilidad inmutable que garantiza trazabilidad, integridad de los datos y auditoría. Se trata de la Blockchain. En ella, cada

miembro de la red, aparte de realizar el entrenamiento por su cuenta, ahora necesita estar autenticado en la red de Blockchain y dejará siempre una huella en el sistema en forma de registro. Esta información se almacenará en el sistema y se usará para su cotejo en cualquier momento del entrenamiento, evitando así falsificaciones y corroborando que la información recibida por todos los miembros de la red es verídica.

1.3 Dilema de seguridad y rendimiento

El principal desafío de la integración de este tipo de red aparece en el equilibrio entre la robustez del sistema y su eficiencia. Por un lado, la tecnología Blockchain introduce una capa de seguridad importante mediante el consenso y la inmutabilidad y, por otro, añade una sobrecarga de computación y latencia de red (overhead).

En el caso de sistemas ejecutados en tiempo real (sistemas críticos) como lo son los vehículos autónomos, las actualizaciones del modelo deben ser rápidas y el hardware tiene limitaciones de consumo energético. Por ello, es necesario determinar si es asumible el tiempo adicional que requiere el procesamiento con la red de Blockchain. El dilema se resume en la siguiente pregunta: ¿cuánta latencia estamos dispuestos a sacrificar a cambio de una red de entrenamiento auditable y más protegida? Los resultados se expondrán a lo largo de esta memoria.

1.4 Motivación del proyecto: Validación de una arquitectura Blockchain-IA híbrida

Este Trabajo de Fin de Grado surge de la necesidad de validar y extender unas investigaciones recientes que proponen el uso de la tecnología Blockchain como pilar fundamental de seguridad y auditoría en los sistemas distribuidos. Las investigaciones que tratan de resolver esta problemática y de la que surge el presente trabajo son las siguientes:

Verificación de eventos en tiempo real (Pujol y cols., 2024): Este proyecto utiliza una red Hyperledger Besu y el algoritmo de consenso IBFT 2.0 para la detección de diferentes situaciones en la carretera. Con esta investigación se demuestra que un sistema descentralizado puede reaccionar ante diferentes situaciones entre 0.32 y 0.82 segundos, reduciendo de manera importante el tiempo de reacción de los seres humanos, que es de 0.5 a 1 segundo.

Trabajo previo del grupo de investigación sobre aprendizaje federado y Blockchain (CloudLab-AIA, 2026): Este trabajo previo en preparación realizado por el mismo grupo de investigación anterior integra el Aprendizaje Federado con Blockchain para permitir un entrenamiento de modelos de IA más seguro. En esta propuesta se define el problema de la falta de confianza entre los miembros de la red del entrenamiento distribuido, usando contratos inteligentes para el registro de secuencias criptográficas (hashes) de los modelos, garantizando la auditabilidad de cada ronda del entrenamiento.

El resultado de combinar ambas investigaciones se puede observar en la figura 1.1, que muestra de manera visual el panorama general y el punto de partida de este proyecto. El

objetivo de este Trabajo de Fin de Grado es, por tanto, validar la propuesta de la segunda investigación, la de integrar Blockchain en el proceso de entrenamiento del modelo distribuido.

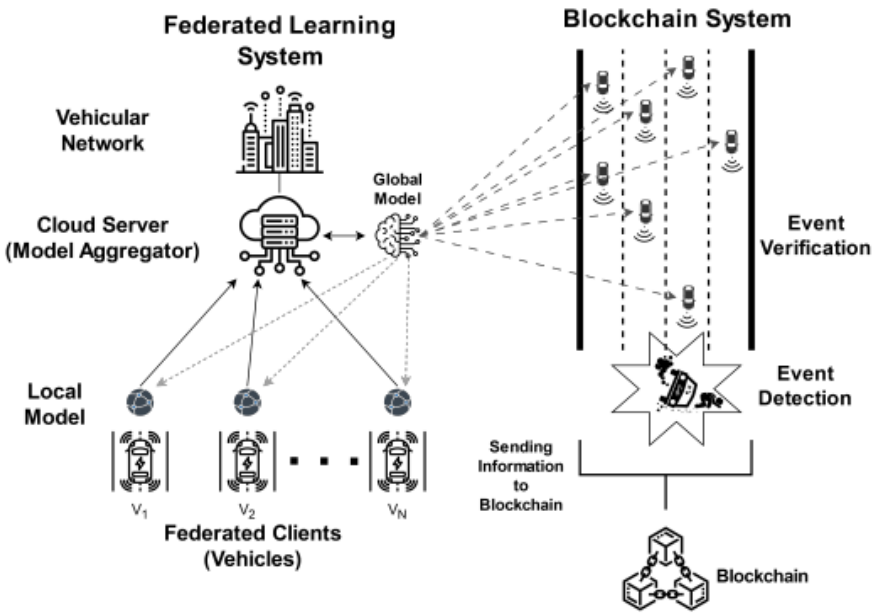


Figura 1.1: Arquitectura a validar

2 Marco Teórico

2.1 Blockchain

2.1.1 Concepto de Blockchain

Una *Blockchain* (cadena de bloques) es una estructura de datos distribuida y descentralizada (Nakamoto, 2008; Zheng y cols., 2018) que permite almacenar información de forma inmutable y verificable, sin necesidad de una entidad (o autoridad) central que la gestione. De manera conceptual, la Blockchain puede entenderse como un libro de contabilidad (*ledger*) compartido entre múltiples participantes, en el que todos los nuevos registros quedan enlazados criptográficamente con el anterior, formando una cadena que resulta extremadamente difícil de alterar.

La información se agrupa en **bloques**. Cada bloque contiene un conjunto de transacciones, una marca de tiempo y, lo más importante y fundamental, el *hash* criptográfico del bloque anterior. Este encadenamiento de hashes es lo que da nombre a la tecnología y constituye su principal garantía de integridad: si se modifica el contenido de un bloque ya existente, cambia su hash, por lo que el enlace con todos los bloques posteriores se rompe y haría visible por tanto una evidente manipulación. Para alterar un único registro sería necesario recalcular todos los bloques siguientes y, además, lograr que la mayoría de la red aceptara esa versión modificada, algo que es computacionalmente inviable en una red sana y activa.

El verdadero valor de la Blockchain no reside únicamente en la estructura de datos, sino en el problema que resuelve: el de la **confianza entre partes que no confían entre sí**. En los sistemas tradicionales, cuando varias entidades necesitan compartir información, suelen depender de una entidad intermediaria de confianza (un banco, una autoridad de registro, un servidor central, etc...) la cual custodia y valida los datos. La Blockchain elimina esa necesidad haciendo que la confianza ya no se deposita en un tercero, sino en el propio sistema y en los mecanismos criptográficos y de consenso que lo mantienen. Todos los participantes comparten una misma versión de la verdad, y ninguno puede modificarla unilateralmente.

De este modo, la Blockchain aporta tres propiedades especialmente relevantes para este proyecto. Por un lado está la **integridad de los datos**, donde una vez registrada la información, esta no puede modificarse ni eliminarse sin que el resto de la red lo detecte. La **trazabilidad**, que indica que cada registro queda asociado a su autor y a un instante temporal, lo que permite reconstruir el historial completo de operaciones. Por útlimo, la **descentralización**, que define que no existe un punto único de control ni de fallo, ya que la información se replica entre todos los participantes de la red.

Estas características hacen de la Blockchain una tecnología perfecta para escenarios en los

que la confianza, la auditoría y la integridad de la información son requisitos esenciales, como es el caso del entrenamiento distribuido de modelos de Inteligencia Artificial, que es lo que se aborda en este trabajo.

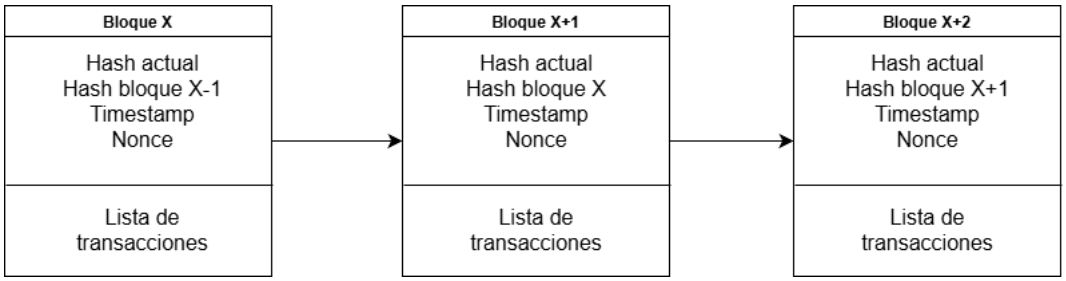


Figura 2.1: Encadenamiento de bloques en red Blockchain

2.1.2 Funciones hash criptográficas

Una **función hash criptográfica** es un algoritmo que transforma una entrada de datos de tamaño indeterminado en una salida de tamaño fijo, denominado *hash* o resumen (*digest*). Estas funciones son una pieza fundamental de la tecnología Blockchain y, en el caso de este proyecto, también del sistema de almacenamiento distribuido empleado.

Para ser consideradas criptográficamente seguras, estas funciones deben cumplir una serie de propiedades (Menezes y cols., 1996), como son el **determinismo**, que significa que una misma entrada produce siempre el mismo hash. La **eficiencia**, que indica que el cálculo del hash es rápido para cualquier entrada. La **resistencia a preimagen**, que significa que dado un hash, es computacionalmente inviable obtener la entrada que lo generó. El **efecto avalancha**, indica que un cambio mínimo en la entrada (incluso de un solo bit) produce un hash completamente distinto. por último, la **resistencia a colisiones**, que afirma que es inviable encontrar dos entradas distintas que produzcan el mismo hash.

Estas propiedades hacen que el hash funcione como una especie de **huella digital** del contenido, identificando de manera unívoca un conjunto de datos, de modo que cualquier alteración, por pequeña que sea, resulta inmediatamente detectable al cambiar por completo el hash resultante.

En el caso y contexto de este proyecto, las funciones hash desempeñan un papel doble. Por un lado, son el mecanismo que encadena los bloques de la Blockchain y garantiza su inmutabilidad. Por otro lado, son la base del direccionamiento por contenido del sistema IPFS, en el que el identificador de un archivo (su CID) se deriva directamente de su contenido mediante una función hash. Gracias a esta propiedad, el sistema puede verificar que un modelo almacenado coincide exactamente con el que fue registrado, sin más que comparar sus hashes. Una de las familias de funciones hash más utilizadas, y empleada tanto en Ethereum como en IPFS, es **SHA-256** (National Institute of Standards and Technology, 2015), que produce resúmenes de 256 bits.

2.1.3 Tipos de redes Blockchain

No todas las redes Blockchain son iguales. En función de quién puede participar en ellas y con qué permisos, se distinguen tres tipos: públicas, privadas y de consorcio (Zheng y cols., 2018). Esta clasificación es relevante para este proyecto, ya que la elección del tipo de red condiciona aspectos como la privacidad, el rendimiento y el control sobre los participantes.

Redes públicas (sin permisos)

En una red blockchain pública o *permissionless*, cualquier persona puede unirse a la red, leer su contenido, enviar transacciones y participar en el proceso de consenso sin necesidad de autorización previa. Son redes completamente abiertas y descentralizadas, como Bitcoin o Ethereum en su red principal. Su principal fortaleza es la ausencia total de control central, pero presentan inconvenientes notables para ciertos casos de uso, que se traduce en un rendimiento limitado (baja cantidad de transacciones por segundo), una elevada latencia y, normalmente, un coste por transacción asociado al consumo de recursos de la red.

Redes privadas (con permisos)

En una red privada o *permissioned*, el acceso está restringido a un conjunto de participantes previamente autorizados (Hyperledger Besu, s.f.). Una entidad o administrador controla quién puede unirse a la red, leer la información y validar transacciones. Este modelo sacrifica parte de la descentralización a cambio de un mayor control, un rendimiento muy superior y la posibilidad de eliminar los costes por transacción. Son adecuadas para entornos empresariales o controlados en los que los participantes son conocidos.

Redes de consorcio

Las redes de consorcio son un punto intermedio entre las públicas y las privadas. En ellas, el control no recae sobre una única entidad, sino sobre un grupo de organizaciones que gestionan la red de forma conjunta. Cada miembro del consorcio participa en el consenso y en la administración, lo que las hace perfectas para escenarios de colaboración entre varias entidades que no confían en su totalidad entre sí pero comparten un objetivo común.

En la siguiente tabla 2.1 se puede ver un resumen de las características de cada tipo de red.

Característica	**Pública**	**Privada**	**Consorcio**
Acceso	Abierto a cualquiera	Restringido	Restringido a miembros
Control	Descentralizado	Una entidad	Grupo de entidades
Permisos	Sin permisos	Permisionada	Permisionada
Rendimiento	Bajo	Alto	Alto
Coste por transacción	Sí	Configurable (nulo)	Configurable (nulo)
Descentralización	Total	Baja	Media
Ejemplo	Bitcoin, Ethereum	Hyperledger Besu	Redes interbancarias

Tabla 2.1: Comparativa de los tipos de redes Blockchain.

Elección para este proyecto

Para este trabajo se ha optado por una red **permisionada** (privada), ya que encaja con el escenario de aprendizaje federado planteado, donde se indica que los participantes del entrenamiento son conocidos y deben estar autenticados, no existe necesidad de abrir la red a usuarios anónimos, y resulta esencial disponer de un alto rendimiento y de un coste por transacción nulo para que el registro de cada ronda de entrenamiento sea viable. Una red pública, con su latencia y sus costes asociados, haría inviable el sistema, mientras que una red permisionada proporciona el control y la eficiencia necesarios sin renunciar a las garantías de integridad y trazabilidad propias de la tecnología Blockchain.

2.1.4 Elementos de la red

Una red Blockchain se compone de una serie de elementos que trabajan de forma coordinada para mantener el funcionamiento y la integridad del sistema. A continuación se describen los más relevantes para este proyecto.

2.1.4.1 Nodo

Un **nodo** representa a cada una de las máquinas o entidades que participan en una red Blockchain ejecutando su software. Los nodos son los elementos básicos que forman la red: se comunican entre sí, intercambian transacciones y bloques y mantienen cada uno una copia propia del estado de la cadena. Esta replicación de la información en diferentes nodos es lo que le proporciona a la Blockchain su naturaleza descentralizada y su resistencia a fallos, ya que no existe un punto único cuya caída comprometa al sistema completo.
No todos los nodos desempeñan el mismo papel. Según las funciones que asumen, pueden distinguirse varios tipos (Ethereum Foundation, s.f.-d), como son los siguientes:

- **Nodos validadores**: participan en el proceso de consenso, proponiendo y validando nuevos bloques. Son los responsables de decidir qué transacciones se añaden a la cadena.
- **Nodos completos** (*full nodes*): mantienen una copia completa de la cadena y verifican las transacciones y bloques, pero no necesariamente tienen que participar en el consenso.
- **Nodos de acceso o RPC**: actúan como punto de entrada para las aplicaciones externas, exponiendo interfaces (como JSON-RPC) a través de las cuales los clientes pueden consultar el estado de la cadena o enviar transacciones, sin participar ellos mismos en la validación.

En la red desplegada para este proyecto existen nodos de estos roles comentados, los cuales conviven para mantener la red activa. Concretamente son los siguientes: un conjunto de nodos validadores encargados del consenso y un nodo RPC que sirve como punto de conexión para el sistema de aprendizaje federado.

2.1.4.2 Red y Peers

Los nodos de una Blockchain se organizan en una red **peer-to-peer** (P2P) (Ethereum Foundation, s.f.-c), un modelo de comunicación en el que todos los participantes tienen el

mismo rango y se conectan directamente entre sí, sin depender de un servidor central que coordine los intercambios. Cada nodo de la red se denomina **peer** (par), y se comunica con un subconjunto de otros peers a los que está conectado directamente.

En este tipo de red, cuando un nodo genera una nueva transacción o un nuevo bloque, lo propaga a sus peers, que a su vez lo reenvían a los suyos, hasta que la información se difunde por toda la red. Este mecanismo de propagación, conocido como *gossip*, garantiza que todos los nodos acaben recibiendo la misma información sin necesidad de un punto central de distribución.

Para que un nodo pueda incorporarse a la red y comenzar a comunicarse, necesita conocer la dirección de al menos algunos de los nodos ya existentes. Estos nodos de entrada se conocen como **bootnodes**, y actúan como punto inicial de contacto a partir del cual el nuevo nodo descubre al resto de la red. Cada nodo se identifica de forma única mediante un **enode**, una dirección que combina su clave pública con su dirección IP y puerto, y que permite a los demás peers localizarlo y establecer conexión con él.

Esta arquitectura P2P es la que mantiene la descentralización de la Blockchain, ya que al no existir un servidor central, la red sigue funcionando aunque algunos nodos se desconecten, y ningún participante individual tiene control sobre la comunicación del conjunto.

2.1.4.3 Consenso

En una red descentralizada, donde varios nodos mantienen cada uno su propia copia de la cadena y no existe una autoridad central que mantenga una única versión o fuente de la verdad, surge un problema fundamental: ¿cómo se ponen de acuerdo todos los nodos sobre cuál es el estado válido de la Blockchain? Es decir, ¿cómo se decide qué transacciones son verídicas y en qué orden se añaden los bloques, garantizando que todos los participantes compartan exactamente la misma versión de la cadena? La respuesta es el **mecanismo de consenso**.

Un algoritmo de consenso es el conjunto de reglas mediante el cual los nodos de la red acuerdan, de forma coordinada y sin confiar unos en otros, qué bloque se añade a continuación a la cadena. Su objetivo es mantener la coherencia del sistema incluso habiendo nodos que fallan o que se comportan de manera maliciosa. Este último escenario se conoce como el **problema de los generales bizantinos** (Lamport y cols., 1982), una situación clásica en sistemas distribuidos en la que algunos participantes pueden enviar información falsa o contradictoria, y frente a la cual un buen algoritmo de consenso debe ser capaz de seguir funcionando correctamente. Los algoritmos que toleran este tipo de comportamiento se denominan **tolerantes a fallos bizantinos** (BFT, *Byzantine Fault Tolerant*).

Existen numerosos algoritmos de consenso, cada uno con sus ventajas e inconvenientes en cuanto a seguridad, rendimiento y grado de descentralización. Algunos de los más relevantes son:

Proof of Work (PoW)

Es el mecanismo empleado originalmente por Bitcoin. Los nodos (llamados mineros) compiten por resolver un problema criptográfico costoso computacionalmente (Ethereum Foundation, s.f.-a), y el primero que lo consigue obtiene el derecho a añadir el siguiente bloque (recibiendo una recompensa por ello). Es muy seguro y robusto, pero extremadamente ineficiente en términos de consumo energético y de rendimiento, lo que lo hace inadecuado para redes permisionadas.

Proof of Stake (PoS)

En lugar de competir mediante poder de cómputo, los validadores son seleccionados para proponer bloques en función de la cantidad de criptomoneda que poseen y "bloquean" como garantía (*stake*). Reduce drásticamente el consumo energético respecto a PoW, y es el mecanismo al que ha migrado Ethereum en su red principal.

Algoritmos de tipo BFT

Diseñados para redes con un número conocido y limitado de validadores, como las redes permisionadas. En ellos, los validadores se comunican entre sí para acordar cada bloque mediante rondas de votación, ofreciendo un alto rendimiento y que un bloque una vez confirmado no puede revertirse. A esta familia pertenecen algoritmos como IBFT, QBFT o Raft, y es la más adecuada para el escenario de este proyecto.

Elección para este proyecto

Dado que la red de este trabajo es permisionada y cuenta con un número determinado y reducido de validadores, los mecanismos basados en competición (PoW, PoS) son demasiado ineficientes para el escenario que se plantea en este TFG, lo que da lugar al empleo de un algoritmo de tipo BFT. Concretamente es el IBFT 2.0, que se describe en detalle en el apartado siguiente, el cual proporciona la finalidad inmediata (un bloque confirmado no puede revertirse) y el rendimiento que requiere el registro de las rondas de entrenamiento.

2.1.4.4 IBFT 2.0

El algoritmo de consenso usado en la red de este proyecto es **IBFT 2.0** (*Istanbul Byzantine Fault Tolerant*), que es un protocolo de tipo BFT diseñado para redes permisionadas y utilizado por defecto en la red generada con Hyperledger Besu. Se trata de la evolución del algoritmo IBFT original, con mejoras en su seguridad y en la gestión de la pertenencia de los validadores.

En IBFT 2.0, un conjunto de nodos previamente autorizados (los **validadores**), son los responsables de proponer y validar los bloques (Moniz, 2020). El funcionamiento se organiza en rondas: en cada ronda, uno de los validadores es seleccionado como proponente (siguiendo un esquema rotatorio de tipo *round-robin*) y se encarga de crear el siguiente bloque y difundirlo al resto. Los demás validadores verifican la propuesta y, a través de un proceso de votación en varias fases, acuerdan si el bloque debe añadirse a la cadena. Un bloque solo se

considera válido cuando ha sido firmado por una śupermayoríađe validadores, concretamente al menos dos tercios del total.

Este requisito de la supermayoría es lo que dota al algoritmo de su tolerancia a fallos bizantinos. IBFT 2.0 garantiza el funcionamiento correcto de la red siempre que el número de validadores defectuosos o maliciosos f cumpla la relación con el número total de validadores n:

$$n \geq 3f + 1$$

Es decir, para tolerar el fallo de un validador se necesitan al menos cuatro; para tolerar dos, al menos siete. Esto explica que la configuración por defecto de la red parta de cuatro validadores, capaces de tolerar el fallo de uno de ellos.

Una de las propiedades más valiosas de IBFT 2.0 para este proyecto es la **finalidad inmediata**: una vez que un bloque es acordado y añadido a la cadena, queda confirmado de forma definitiva y no puede revertirse mediante bifurcaciones (*forks*), a diferencia de lo que ocurre en algoritmos probabilísticos como Proof of Work. Esta característica es fundamental en el contexto del registro de las rondas de entrenamiento, ya que garantiza que una vez registrada una contribución o un modelo agregado, ese registro es permanente e irrefutable.

Además, IBFT 2.0 permite gestionar dinámicamente el conjunto de validadores: es posible añadir o eliminar validadores de la red en funcionamiento mediante un proceso de votación entre los validadores existentes, sin necesidad de detener la red ni de reiniciar la cadena.

2.1.4.5 Validadores

Los validadores son nodos que tienen autoridad para participar en el proceso de consenso de la red. Un nodo cualquiera se limita a mantener una copia de la cadena o puede servir como punto de acceso, pero solo los validadores tienen la capacidad de proponer nuevos bloques y de votar sobre la validez de los bloques propuestos por otros. Son, por tanto, los responsables últimos de decidir qué información se incorpora a la Blockchain.

En una red permisionada como la de este proyecto, el conjunto de validadores está definido y está controlado: no cualquier nodo puede convertirse en validador, sino que debe ser autorizado e incorporado al conjunto mediante el procedimiento de votación que establece el algoritmo de consenso. Esta restricción es precisamente lo que distingue a una red permisionada y lo que garantiza que solo participantes de confianza intervengan en la validación. (Hyperledger Besu, s.f.)

Cada validador dispone de un par de claves criptográficas propio. La **clave privada** se utiliza para firmar los bloques y los votos que emite, de modo que su participación en el consenso queda criptográficamente acreditada y no puede ser suplantada. La **clave pública** (y la dirección que deriva de ella) identifica al validador dentro de la red. Gracias a este esquema de firmas, cualquier nodo puede verificar que un bloque ha sido efectivamente acordado por los validadores legítimos, sin posibilidad de falsificación.

El número de validadores de una red es un elemento de diseño importante, ya que determina su tolerancia a fallos y su rendimiento. Un mayor número de validadores aumenta la robustez de la red y la cantidad de nodos defectuosos que puede soportar, pero también aumenta la comunicación necesaria para alcanzar el consenso y, con ello, la latencia. Este balance entre robustez y rendimiento se analiza experimentalmente en este trabajo comparando configuraciones con distinto número de validadores.

2.1.4.6 Gas

El **gas** es la unidad que mide el coste computacional de ejecutar operaciones en una Blockchain (Ethereum Foundation, s.f.-b). Cada operación que un nodo debe procesar (una transferencia, el almacenamiento de un dato, la ejecución de una instrucción de un smart contract, etc...) tiene asignado un coste en gas que representa la cantidad de recursos que consume. De este modo, el gas funciona como una medida objetiva del trabajo que una transacción supone a la red.

En el caso de las redes públicas, el uso del gas supone una importancia doble. Por un lado, sirve para **retribuir** a los nodos que procesan y validan las transacciones, ya que el emisor paga una cantidad proporcional al gas consumido. Y, por otro lado, actúa como mecanismo de **protección frente a abusos** ya que al tener cada operación un coste, se desincentiva la ejecución de código malicioso o de bucles infinitos que podrían saturar la red, pues agotarían el gas disponible y la transacción se detendría.

El coste total de una transacción se determina por dos factores que son la cantidad de gas que consume (*gas used*), el cual depende de la complejidad de la operación, y el precio que se paga por cada unidad de gas (*gas price*). Un detalle relevante del modelo de coste es que la primera escritura de un valor en una posición de almacenamiento resulta más cara que las posteriores modificaciones de ese mismo valor, ya que inicializar un espacio de almacenamiento previamente vacío consume más recursos que actualizarlo.

En una red **permisionada** como la de este proyecto, donde no existe criptomoneda ni incentivo económico, el *gas price* se configura a cero, por lo que las transacciones no tienen coste monetario alguno. Sin embargo, el gas no pierde su función ya que sigue siendo una métrica objetiva del coste computacional de cada operación, independiente del hardware o de la carga puntual de la red. Por esta razón, en este Trabajo de Fin de Grado se usa el gas consumido por las transacciones del *smart contract* como una de las variables de medición, ya que permite cuantificar de forma precisa y reproducible el peso computacional que cada operación de registro impone a la red Blockchain.

2.1.5 Máquina Virtual de Ethereum (EVM)

La **Máquina Virtual de Ethereum** (EVM, *Ethereum Virtual Machine*) es el entorno de ejecución en el que se ejecutan los *smart contracts* dentro de una Blockchain compatible con Ethereum. Puede entenderse como un ordenador virtual, replicado e idéntico en todos los nodos de la red, el cual ejecuta el código de los contratos de forma determinista: ante una misma entrada y un mismo estado de la cadena, la EVM produce siempre el mismo resultado en cualquier nodo. Esta propiedad es esencial, ya que garantiza que todos los participantes lleguen al mismo estado tras procesar las mismas transacciones. Esto es imprescindible para alcanzar el consenso. (Wood, 2014)

El código de los *smart contracts* no se ejecuta directamente en el lenguaje en el que se escribe (habitualmente Solidity), sino que primero se **compila** a un conjunto de instrucciones de bajo nivel denominadas *bytecode*, que es lo que la EVM interpreta y ejecuta. Cada una de estas instrucciones (*opcodes*) tiene asociado un coste en gas, de manera que la EVM lleva la cuenta de los recursos consumidos durante la ejecución y detiene el proceso si se agota el gas asignado a la transacción.

Una de las grandes ventajas de la EVM es que se ha convertido en un estándar es que numerosas plataformas de Blockchain (aparte de la propia red de Ethereum) han adoptado la compatibilidad con la EVM. Esto significa que un mismo *smart contract*, escrito en Solidity y compilado a *bytecode*, puede desplegarse y ejecutarse en cualquiera de estas plataformas sin que sea necesario modificarlo. Tanto Hyperledger Besu como GoQuorum, las plataformas consideradas en este proyecto, son compatibles con la EVM, lo que garantiza que el contrato desarrollado pueda funcionar sobre cualquiera de ellas.

2.1.6 Smart Contracts

Un **smart contract** (contrato inteligente) es un programa que se almacena y se ejecuta sobre la Blockchain. A diferencia de un programa tradicional, que se ejecuta en un servidor bajo el control de una entidad, un *smart contract* se ejecuta de forma descentralizada en la EVM de todos los nodos de la red, y su código y su estado quedan registrados de forma inmutable en la cadena. El término ”contrato” proviene de su capacidad para hacer cumplir automáticamente un conjunto de reglas, sin necesidad de un intermediario que las supervise. Una vez desplegado, el contrato ejecuta exactamente lo que su código establece, y nadie puede alterarlo. (Solidity Authors, s.f.)

Los *smart contracts* se escriben habitualmente en **Solidity**, un lenguaje de programación de alto nivel orientado a contratos y de sintaxis similar a la de lenguajes como JavaScript o C++. Una vez escrito, el contrato se compila a *bytecode* y se despliega en la red mediante una transacción, tras lo cual queda asociado a una **dirección** única que lo identifica. A partir de ese momento, cualquier participante autorizado puede interactuar con él invocando sus funciones a través de esa dirección.

Las funciones de un *smart contract* pueden ser de dos tipos según su efecto sobre el estado de la cadena. Las funciones de **escritura** modifican el estado almacenado en el contrato

(por ejemplo registrar un nuevo dato), por lo que requieren el envío de una transacción, consumen gas y deben ser confirmadas por la red mediante el consenso. Las funciones de **lectura** (llamadas *view* en Solidity) únicamente consultan el estado sin modificarlo, por lo que pueden ejecutarse de forma local en un nodo, sin generar una transacción ni consumir gas.

Otro elemento importante de los *smart contracts* son los **eventos** (*events*). Un evento es un mecanismo mediante el cual un contrato puede emitir una notificación cuando ocurre algo relevante. La información del evento queda registrada en los *logs* de la transacción, accesibles desde fuera de la cadena pero sin ocupar espacio en el almacenamiento del contrato, lo que los hace mucho más económicos en gas que una escritura convencional. Las aplicaciones externas pueden **suscribirse** a estos eventos para reaccionar a ellos en tiempo real, sin necesidad de consultar repetidamente el estado del contrato.

En este proyecto, los *smart contracts* conforman el componente que conecta la red Blockchain con el sistema de aprendizaje federado: un contrato específico se encarga de registrar las contribuciones de cada participante y los modelos agregados de cada ronda, haciendo uso de funciones de escritura para el registro, funciones de lectura para la verificación y eventos para la notificación, tal como se detalla en el capítulo de desarrollo.

2.1.7 Plataformas empresariales: GoQuorum y Hyperledger Besu

La red principal de Ethereum es pública pero su tecnología ha dado se ha utilizado en plataformas orientadas a entornos **empresariales y permisionados**, los cuales conservan la compatibilidad con la EVM pero añaden características como el control de acceso, la privacidad de transacciones y algoritmos de consenso adecuados para redes de participantes conocidos. Dos de las plataformas más relevantes en este ámbito, ambas vinculadas a ConsenSys, son GoQuorum y Hyperledger Besu, que son las consideradas en este proyecto. (ConsenSys, s.f.-a)

GoQuorum

GoQuorum es una plataforma de Blockchain empresarial desarrollada originalmente por JP Morgan y posteriormente mantenida por ConsenSys. Se trata de una bifurcación (*fork*) de *go-ethereum* (la implementación de Ethereum en lenguaje Go), a la que se añaden funcionalidades específicas para entornos privados, como son la privacidad de transacciones y los mecanismos de consenso permisionados. Como deriva directamente de go-ethereum, mantiene su madurez y su compatibilidad con el ecosistema de Ethereum.

Hyperledger Besu

Hyperledger Besu es un cliente de Ethereum de código abierto, escrito en Java y desarrollado por la Hyperledger Foundation (perteneciente a la Linux Foundation). A diferencia de GoQuorum, Besu no es un fork de go-ethereum, sino una implementación independiente, y puede operar tanto en la red pública de Ethereum como en redes privadas permisionadas. Ofrece soporte para varios algoritmos de consenso (entre ellos IBFT 2.0 y QBFT), control

de acceso a la red y privacidad de transacciones, lo que lo hace especialmente adecuado para entornos empresariales.

Comparativa y elección

Ambas plataformas son compatibles con la EVM, por lo que un mismo *smart contract* escrito en Solidity puede ejecutarse en cualquiera de las dos sin modificación, y ambas comparten herramientas del ecosistema, como el gestor de transacciones privadas Tessera. Las principales diferencias residen en su origen e implementación: GoQuorum está escrito en Go y deriva de go-ethereum, mientras que Besu está escrito en Java y es una implementación independiente mantenida por la Hyperledger Foundation.

Para este proyecto se ha optado por **Hyperledger Besu**, por varios motivos como son su carácter abierto bajo licencia Apache 2.0, su respaldo por parte de la Hyperledger Foundation, su soporte nativo de los algoritmos de consenso más recientes y la calidad y claridad de su documentación oficial. Además, el despliegue se ha facilitado mediante el **Quorum Developer Quickstart** (ConsenSys, s.f.-b), una herramienta proporcionada por ConsenSys que permite generar de forma automatizada una red Blockchain permisionada lista para ser usada, pudiendo elegir entre Besu y GoQuorum como cliente y activar opcionalmente el soporte de transacciones privadas, que en el caso de este proyecto se ha optado por activarla.

2.1.8 Privacidad con Tessera

La información registrada por defecto en Blockchain es visible para todos los participantes de la red. Esto es deseable para garantizar la transparencia ya que en muchas situaciones empresariales existe información sensible que solo debería ser accesible para un subconjunto de los participantes. Para cubrir esta necesidad, plataformas como Besu y GoQuorum incorporan el concepto de **transacciones privadas**, gestionadas mediante **Tessera**.

Tessera es un gestor de transacciones (*transaction manager*) de código abierto que se despliega junto a cada nodo de la red. Este gestor trata de separar la información privada de la cadena pública: cuando un nodo envía una transacción privada destinada a uno o varios participantes concretos, el contenido de la transacción se cifra y se almacena fuera de la cadena (*off-chain*) en las instancias de Tessera de los destinatarios autorizados, mientras que en la Blockchain pública únicamente se registra un *hash* que actúa como referencia verificable. De esta manera, los nodos que no figuran entre los destinatarios no pueden acceder al contenido real de la transacción, lo que permite combinar la trazabilidad de la Blockchain con la confidencialidad de cierta información. (ConsenSys, s.f.-c)

En este proyecto, la red desplegada mediante el Quorum Developer Quickstart incorpora Tessera por defecto, por lo que cada nodo cuenta con su correspondiente gestor de transacciones privadas. No obstante, es importante mencionar que la implementación desarrollada **no utiliza las transacciones privadas de Tessera**. Las transacciones que registran las contribuciones y los modelos agregados son públicas dentro de la red permisionada, ya que su finalidad es precisamente la contraria a la confidencialidad, que es proporcionar un registro

auditable y verificable por todos los participantes.

La confidencialidad del sistema se obtiene por de otras maneras más adecuadas a la tipología del problema. Por un lado, el propio **aprendizaje federado** garantiza que los datos de entrenamiento nunca abandonen cada cliente, manteniendo su privacidad sin necesidad de mecanismos adicionales. Por otro, lo que se registra en la cadena no es el modelo en sí, sino una referencia (su identificador en IPFS, el CID), de modo que la información pesada permanece fuera de la Blockchain. Tessera se mantiene, por tanto, como una entidad disponible en la infraestructura que se podría aprovechar (por ejemplo para compartir métricas sensibles únicamente entre el agregador y un cliente concreto) y se plantea como posible línea de trabajo futuro.

2.1.9 Trazabilidad con Blockchain

La **trazabilidad** es la capacidad de reconstruir el historial completo de las operaciones realizadas sobre un sistema, sabiendo en todo momento qué ocurrió, quién lo hizo y cuándo. Es una de las propiedades más valiosas que aporta la tecnología Blockchain y, en el contexto de este proyecto, constituye uno de los pilares de la solución que se ha propuesto en este proyecto. (Zheng y cols., 2018)

La trazabilidad en una Blockchain surge de la combinación de varias de sus características fundamentales. En primer lugar, la **inmutabilidad**: una vez que una operación se registra en la cadena, no se puede modificar ni eliminar, gracias al encadenamiento de bloques mediante hashes y al mecanismo de consenso. En segundo lugar, la **atribución**: cada transacción está firmada con la clave privada de quien la emite, de modo que queda asociada de forma inequívoca a su autor, sin posibilidad de suplantación. Y en tercer lugar, el **orden temporal**: cada operación queda registrada con una marca de tiempo y en una posición determinada de la cadena, lo que permite establecer la secuencia exacta en que se produjeron los acontecimientos.

De la atribución se puede obtener además una propiedad muy relacionada con ella: el **no-repudio**. Dado que cada operación queda firmada criptográficamente por su autor, este no puede negar posteriormente haberla realizado. Esta garantía es especialmente relevante en entornos donde varias partes interactúan sin confiar plenamente entre sí, ya que impide que un participante intente evadirse de la responsabilidad de sus acciones. (Menezes y cols., 1996)

En un sistema de entrenamiento distribuido, las propiedades mencionadas que proporciona la Blockchain resuelven una carencia fundamental del aprendizaje federado clásico, en el que las contribuciones de los participantes se intercambian sin dejar un rastro verificable. Al registrar en la Blockchain cada contribución de cada cliente y cada modelo agregado, se obtiene un historial completo e inmutable del proceso de entrenamiento, es decir, es posible ver qué cliente participó en cada ronda, qué aportó y en qué momento, al igual que qué modelo global resultó de cada agregación. Este registro permite auditar el entrenamiento cuando éste termine y detectar comportamientos anómalos, verificando las entidades que han realizado posibles manipulaciones en el sistema y acuando en consecuencia. Esto dota el sistema de un

nivel de confianza y transparencia que un esquema centralizado no puede ofrecer por sí solo.

Es importante matizar que la Blockchain garantiza la trazabilidad e integridad del **registro** de las contribuciones, pero no la calidad del contenido de estas. Es decir, asegura que se puede verificar de forma fiable qué modelo aportó cada participante y que ese registro no ha sido alterado, pero no determina por sí sola si el modelo aportado fue entrenado correctamente o de forma maliciosa. La detección de contribuciones de baja calidad o malintencionadas requiere mecanismos adicionales que se construyen sobre esta capa de trazabilidad, y que se plantean como línea de trabajo futuro.

2.2 Almacenamiento distribuido: IPFS

2.2.1 Concepto de IPFS

IPFS (*InterPlanetary File System*) es un protocolo y una red **peer-to-peer** usada para el almacenamiento y la distribución de archivos de forma descentralizada. En el modelo tradicional de la web, los archivos se alojan en servidores concretos y se acceden a través de su ubicación (una URL que indica en qué servidor y ruta se encuentra el recurso). IPFS, en cambio, propone que los archivos se reparten entre los nodos de una red y se acceden por su **contenido**, no por su ubicación. (Benet, 2014)

La motivación de IPFS surge de las limitaciones del almacenamiento centralizado, que son la dependencia de un servidor concreto (que constituye un punto único de fallo), la posibilidad de que un mismo recurso cambie sin que el usuario lo perciba y la dificultad para verificar que un archivo descargado es exactamente el que se esperaba. IPFS aborda estos problemas distribuyendo el contenido entre múltiples nodos y garantizando, usando criptografía, que cada archivo pueda identificarse y verificarse de forma unívoca.

En el contexto de este proyecto, IPFS desempeña un papel muy importante ya que es el sistema encargado de almacenar los modelos de Inteligencia Artificial serializados. Almacenar los modelos completos directamente en la Blockchain sería inviable por el elevado coste que supondría guardar datos de tamaños tan grandes en la cadena, al igual que por la saturación que provocaría el proceso. La solución adoptada consiste en almacenar los modelos en IPFS y registrar en la Blockchain únicamente su identificador, combinando así la capacidad de almacenamiento distribuido de IPFS con la inmutabilidad y trazabilidad de la Blockchain.

2.2.2 Direccionamiento por contenido (CID)

El elemento que hace posible el funcionamiento de IPFS es el **direccionamiento por contenido** (*content addressing*). En lugar de identificar un archivo por el lugar donde está almacenado, IPFS lo identifica por **su contenido**. Para ello, cuando un archivo se añade a la red, se calcula un *hash* criptográfico de su contenido, a partir del cual se genera un identificador único denominado **CID** (*Content Identifier*). (Protocol Labs, s.f.)

Este mecanismo aporta un valor esencial en el desarrollo de este Trabajo de Fin de Grado, ya que dado que el CID se deriva directamente del contenido del archivo, cualquier modificación del contenido, por mínima que sea, **produce un CID completamente distinto**. De ello se desarrollan dos puntos clave. La **verificación de integridad**, ya que al descargar un archivo a partir de su CID, es posible comprobar que el contenido recibido es exactamente el esperado, debido a que su hash debe coincidir con el CID solicitado. Si el archivo hubiera sido modificado, su CID no coincidiría. La segunda es la **deduplicación**, donde dos archivos con contenido idéntico tendrán el mismo CID, por lo que la red no almacena copias redundantes del mismo contenido.

Esta propiedad del direccionamiento por contenido es precisamente la que sustenta el mecanismo de verificación del sistema desarrollado. Al registrar en la Blockchain el CID de un

modelo, se está registrando un **compromiso criptográfico** sobre su contenido, que permite a cualquier participante a descargar el modelo posteriormente desde IPFS y comprobar que se corresponde exactamente con el que fue registrado. El CID actúa así como un puente entre el almacenamiento distribuido de IPFS y el registro inmutable de la Blockchain, permitiendo verificar la integridad de los modelos a lo largo de todo el proceso de entrenamiento.

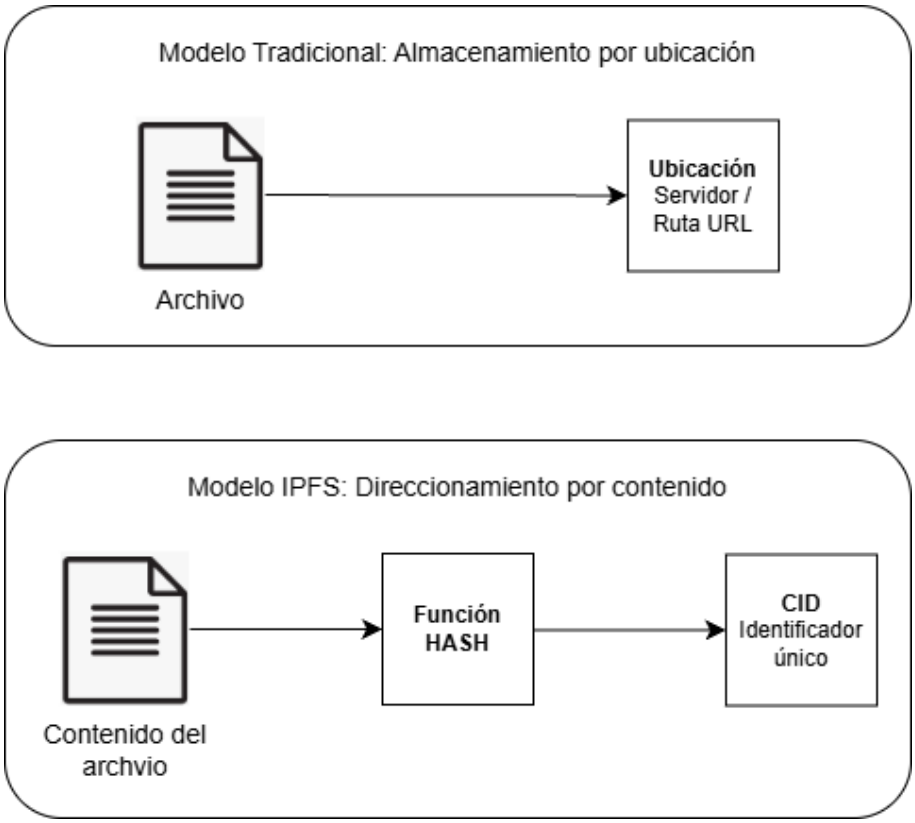


Figura 2.2: Comparativa de almacenamiento de archivos

2.3 Inteligencia Artificial

2.3.1 Concepto de Inteligencia Artificial

La **Inteligencia Artificial** (IA) es la rama de la informática que busca desarrollar sistemas capaces de realizar tareas que normalmente requerirían inteligencia humana, como el reconocimiento de patrones, la toma de decisiones, la clasificación de información, etc... Dentro de la IA, este trabajo se centra en una de sus ramas más relevantes en la actualidad, el aprendizaje automático, y en concreto en su aplicación a un entrenamiento distribuido.

2.3.2 Machine Learning

El **aprendizaje automático** (*Machine Learning*) es una rama de la Inteligencia Artificial en la que los sistemas aprenden a realizar una tarea a partir de datos, sin ser programados explícitamente para ello. En lugar de definir manualmente las reglas, se proporciona al modelo un conjunto de datos a partir del cual ajusta sus parámetros internos hasta ser capaz de generalizar y producir predicciones sobre datos nuevos. Este proceso de ajuste se denomina **entrenamiento**. (Goodfellow y cols., 2016)

En el caso de este proyecto se emplea una **red neuronal**, un tipo de modelo de aprendizaje automático inspirado en la estructura del cerebro, formado por capas de unidades interconectadas que transforman progresivamente los datos de entrada hasta producir una salida. Las redes neuronales son especialmente eficaces en tareas como la clasificación de imágenes, que es el caso de uso abordado en este trabajo. El entrenamiento de una red neuronal consiste en ajustar sus **pesos** (los parámetros de las conexiones entre unidades) de forma iterativa para minimizar el error de sus predicciones.

2.3.3 Aprendizaje Federado

El **aprendizaje federado** (*Federated Learning*) es un paradigma de entrenamiento distribuido en el que múltiples participantes (clientes) entrenan en colaboración un modelo común sin compartir sus datos. A diferencia del entrenamiento centralizado en el que todos los datos se reúnen en un único lugar, en el aprendizaje federado los datos permanecen siempre en cada cliente. Lo que se comparte no son los datos, sino que son los parámetros del modelo entrenado localmente. (McMahan y cols., 2017)

El proceso se organiza en **rondas** coordinadas por un servidor central. En cada ronda, el servidor distribuye el modelo global actual a los clientes, cada cliente lo entrena con sus propios datos locales, y devuelve al servidor los parámetros resultantes. El servidor combina entonces las aportaciones de todos los clientes para obtener un nuevo modelo global mejorado, y el ciclo se repite. De este modo, el conocimiento de todos los participantes se integra en un único modelo sin que ninguno tenga que exponer sus datos.

Este ámbito de la IA aporta dos ventajas fundamentales que motivan su uso en este proyecto. La primera es la **privacidad** ya que al no salir nunca los datos de cada cliente, se preserva su confidencialidad, lo que resulta muy importante cuando se manejan datos sensibles. La segunda es la **distribución de la carga** ya que el entrenamiento se reparte entre

varias máquinas, lo que permite abordar problemas que serían inviables para un único equipo y reduce el coste computacional individual.

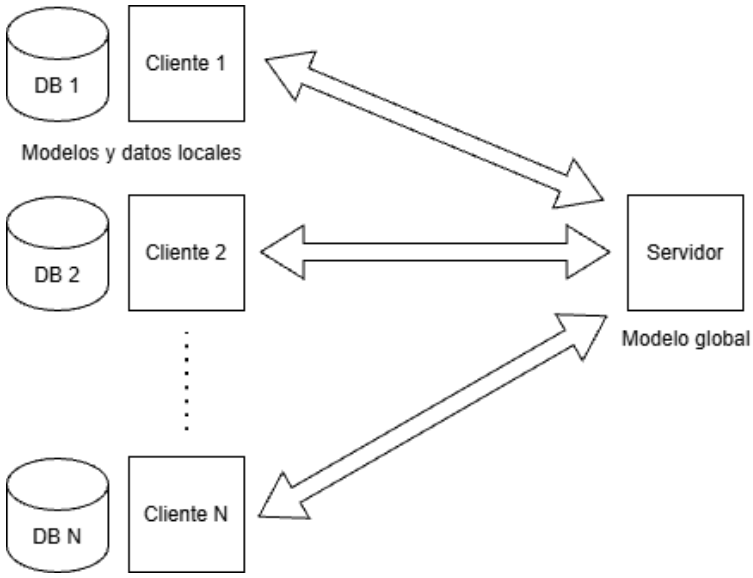


Figura 2.3: Flujo del aprendizaje federado

2.3.4 Algoritmo de agregación: FedAvg

FedAvg (*Federated Averaging*) es el algoritmo de agregación más extendido en aprendizaje federado y es el que se emplea en este proyecto. Su función es combinar los parámetros que los distintos clientes han entrenado localmente en una ronda para producir un único modelo global. (McMahan y cols., 2017)

El algoritmo calcula el nuevo modelo global como una **media ponderada** de los parámetros de los clientes, donde el peso de cada cliente es proporcional al número de muestras con las que ha entrenado. De esta manera, los clientes que disponen de más datos influyen más en el modelo resultante. Los parámetros globales en la ronda $t+1$ se obtienen como:

$$w_{global}^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{(t)}$$

donde $w_k^{(t)}$ son los parámetros del cliente k, n_k el número de muestras de ese cliente, n el número total de muestras entre todos los clientes y K el número de clientes participantes.

Su sencillez y su buen comportamiento en escenarios con datos homogéneamente distribuidos (IID) lo convierten en la opción adecuada para este trabajo, en el que el objetivo no es optimizar el proceso de agregación, sino evaluar la integración del sistema con la capa de trazabilidad basada en Blockchain.

2.3.5 Framework de Flower

Flower (`flwr`) es un *framework* de código abierto especializado en aprendizaje federado, que ha sido usado en este proyecto para implementar y coordinar el entrenamiento distribuido. Su principal ventaja es que abstrae la complejidad de la comunicación entre los participantes, permitiendo centrarse en la lógica del entrenamiento, y es independiente de la librería de aprendizaje automático utilizada, siendo compatible con las principales (como PyTorch, empleada en este trabajo). (Beutel y cols., 2020; Flower Labs, s.f.)

Flower se organiza en torno a dos componentes. Por un lado, el **servidor**, que coordina las rondas de entrenamiento y aplica la estrategia de agregación (en este caso, FedAvg). Por otro, los **clientes**, que implementan la lógica de entrenamiento y evaluación sobre sus datos locales. La comunicación entre ambos se realiza mediante el protocolo **gRPC**, que Flower gestiona de forma transparente para el desarrollador.

Para definir el comportamiento de un cliente, Flower proporciona interfaces que deben implementarse con los métodos básicos del ciclo de aprendizaje federado que son el entrenamiento local del modelo y su evaluación. Del lado del servidor, Flower ofrece estrategias de agregación predeterminadas que pueden utilizarse directamente o extenderse para adaptarlas a necesidades específicas, como se ha hecho en este proyecto para incorporar la verificación contra la Blockchain.

2.3.6 Dataset utilizado

El conjunto de datos empleado para el entrenamiento es un **dataset de imágenes de tráfico**, el cual procede del repositorio FL-Blockchain del grupo CloudLab-AIA (CloudLab-AIA, 2026). Se ha reutilizado en este proyecto para mantener la coherencia con la línea de investigación en la que se enmarca el trabajo.

Este dataset plantea un problema de **clasificación multiclase** con cuatro categorías que representan distintas situaciones que pueden darse en una vía: accidente (`accident`), tráfico denso (`dense_traffic`), condiciones meteorológicas adversas (`meteorology`) y tráfico fluido (`sparse_traffic`). Cada imagen está etiquetada con una de estas cuatro clases, y el conjunto se reparte entre los clientes para el entrenamiento federado. La elección de este caso se da principalmente porque encaja con el escenario que motiva el proyecto, el de los sistemas de asistencia y conducción en entornos viales. Distintos participantes, como vehículos o cámaras de tráfico, podrían disponer de imágenes locales que no es deseable centralizar, pero que pueden contribuir colectivamente a entrenar un modelo común de detección de situaciones en la carretera.

2.4 Integración de Inteligencia Artificial con Blockchain

2.4.1 Problemas del aprendizaje federado

Aunque el aprendizaje federado resuelve el problema de la privacidad de los datos al mantenerlos en cada cliente, introduce nuevas vulnerabilidades derivadas precisamente de su naturaleza distribuida y de la participación de múltiples actores que no necesariamente confían entre sí. Los principales problemas son los siguientes:

El primero es la **falta de trazabilidad y auditoría**. En un esquema federado clásico, las contribuciones de los clientes se envían al servidor y se agregan sin dejar un registro inalterable de quién aportó qué y en qué momento. Esto dificulta auditar el proceso a posteriori y detectar el origen de un posible problema.

El segundo es la **confianza en el servidor central**. El servidor agregador constituye un punto único de confianza: los clientes deben asumir que agrega correctamente las contribuciones y que el modelo global que distribuye es legítimo, sin disponer de un mecanismo para verificarlo.

El tercero es la **vulnerabilidad frente a participantes maliciosos**. Un cliente no autorizado o malintencionado puede intentar enviar contribuciones manipuladas para alterar el comportamiento del modelo global. Sin mecanismos de autenticación y de registro verificable, resulta difícil identificar y atribuir este tipo de comportamientos. Este es precisamente el riesgo que motiva el presente trabajo, ilustrado por el escenario de un participante que tratara de degradar deliberadamente un modelo de detección de situaciones en la carretera.

2.4.2 Uso de Blockchain como solución

La tecnología Blockchain aborda directamente las carencias anteriores, aportando una capa de confianza y verificación sobre el proceso de aprendizaje federado. Su incorporación proporciona, en primer lugar, un **registro inmutable y trazable** de todas las contribuciones: cada aportación de cada cliente y cada modelo agregado queda registrado de forma permanente y verificable, lo que permite auditar el entrenamiento en cualquier momento.

En segundo lugar, aporta **autenticación y no-repudio**: al estar cada transacción firmada criptográficamente por su autor, se garantiza que solo participantes autorizados intervienen en el proceso y que ninguno puede negar sus aportaciones. Y en tercer lugar, reduce la **dependencia del servidor central como único punto de confianza**, ya que el registro queda distribuido y puede ser verificado por cualquier participante de forma independiente.

La combinación de Blockchain con un sistema de almacenamiento distribuido como IPFS permite, además, resolver la limitación del coste de almacenamiento: los modelos se guardan en IPFS y la Blockchain conserva únicamente sus identificadores, manteniendo la integridad del conjunto sin sobrecargar la cadena. De esta forma, la integración de ambas tecnologías dota al aprendizaje federado de las garantías de seguridad, trazabilidad y auditoría de las que carecía, sin renunciar a la privacidad de los datos que lo caracteriza.

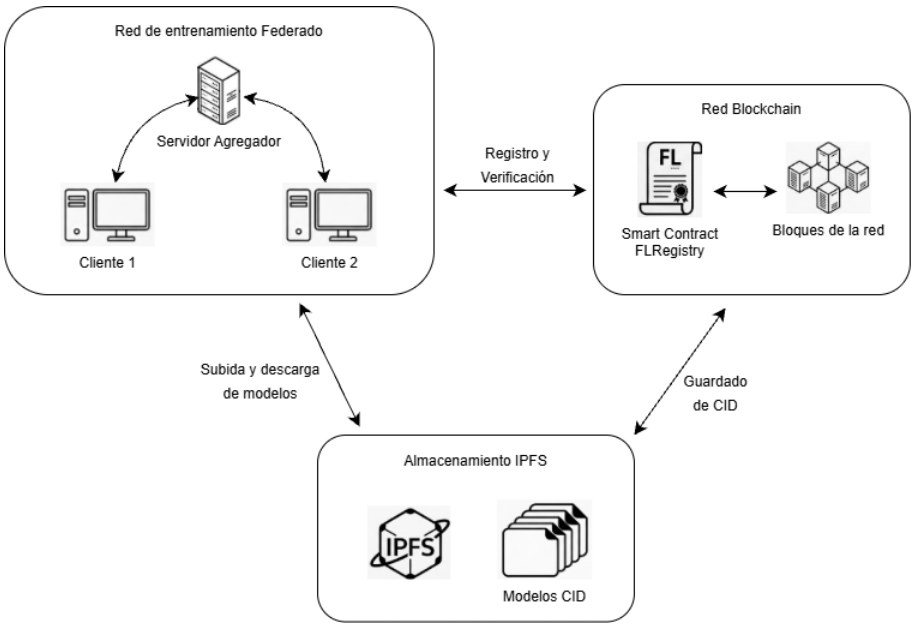


Figura 2.4: Arquitectura del sistema completa

3 Objetivos

A continuación se especificarán los objetivos del el proyecto, diferenciando entre un objetivo general que define la meta global de este trabajo y una serie de objetivos específicos que representan los pasos necesarios para alcanzarlo.

3.1 Objetivo general

El objetivo general de este Trabajo de Fin de Grado es **validar la viabilidad de integrar la tecnología Blockchain en el proceso de entrenamiento de un modelo de Inteligencia Artificial distribuido**, añadiéndole una capa de trazabilidad, auditoría y confianza que permita detectar contribuciones maliciosas, y, además, **determinar si el sobrecoste en latencia y recursos que introduce dicha capa es asumible** para un despliegue en entornos reales, incluidos los sistemas críticos en tiempo real como los vehículos autónomos.

Este trabajo parte de dos líneas de investigación previas, la verificación de eventos en tiempo real sobre una red Hyperledger Besu y la integración de aprendizaje federado con Blockchain para garantizar la auditabilidad del entrenamiento, y busca validar y extender la segunda, combinando ambas en un sistema funcional y evaluable.

3.2 Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. **Estudiar el estado del arte** del aprendizaje federado y de la tecnología Blockchain, así como las investigaciones previas que motivan este trabajo, identificando las limitaciones del entrenamiento distribuido clásico en cuanto a auditoría y trazabilidad de las contribuciones.
2. **Diseñar una arquitectura híbrida** que combine una red Blockchain permisionada con una red de entrenamiento federado, definiendo cómo interactúan ambos sistemas y qué información debe registrarse de manera constante e inmutable para garantizar la auditabilidad de cada ronda.
3. **Desplegar una red Blockchain permisionada** basada en Hyperledger Besu y desarrollar un *smart contract* capaz de registrar las contribuciones de cada participante y los modelos agregados, permitiendo el cotejo de la información en cualquier momento del entrenamiento.
4. **Implementar un sistema de aprendizaje federado** funcional, con un servidor agregador y varios clientes que entrenan de forma conjunta un modelo de clasificación de imágenes de tráfico sin centralizar sus datos.

5. **Integrar ambos sistemas** mediante un mecanismo de almacenamiento distribuido (IPFS) y una verificación cruzada entre cliente y servidor que garantice la integridad de los modelos intercambiados y permita detectar manipulaciones.

6. **Evaluar experimentalmente el sistema** en distintas configuraciones de despliegue, midiendo el sobrecoste temporal y de recursos introducido por la capa Blockchain y analizando su comportamiento al variar el número de máquinas y de validadores.

7. **Responder al dilema entre seguridad y rendimiento**, determinando cuánta latencia adicional supone la auditabilidad que aporta la Blockchain y valorando si dicho coste es asumible en función del tipo de aplicación.

8. **Extraer conclusiones** sobre la viabilidad de la propuesta y proponer líneas de mejora y trabajo futuro.

4 Metodología

En este capítulo se describe la manera en la que se ha trabajado, el enfoque de desarrollo seguido durante la realización del proyecto, las distintas fases en las que se ha estructurado, el conjunto de tecnologías y herramientas empleadas, y la organización del código resultante en el repositorio.

4.1 Enfoque de trabajo

El proyecto se ha abordado siguiendo un enfoque incremental basado en fases y cumplimiento de objetivos. En vez de construir el sistema completo de una sola vez, el trabajo se ha dividido en diferentes etapas sucesivas en las que cada una se apoya en los resultados de la anterior. De esta manera se permite validar cada componente por separado, entendiendo y asimilando los diferentes conceptos y haciendo una construcción del sistema secuencial y funcional.

Esta metodología de trabajo se corresponde con la naturaleza de exploración y de investigación del proyecto, que se basa en la integración de Blockchain con aprendizaje federado, por lo que combina dos tecnologías complejas que funcionan de forma independiente. Es por ello que resultaba esencial asegurar que cada una funcione correctamente por separado antes de unificarlas, al igual que entender los diferentes conceptos a lo largo de la investigación, ya que son temáticas y problemáticas que no han sido tratadas apenas en las clases de la universidad. En resumen, un enfoque de este estilo reduce el riesgo de errores complejos y difíciles de encontrar, facilita la detección temprana de problemas y permite reorientar el trabajo a medida que se adquiere conocimiento sobre las tecnologías implicadas.

4.2 Fases del proyecto

El desarrollo se ha organizado en las siguientes fases.

1. **Investigación y estado del arte**: estudio de los fundamentos teóricos del aprendizaje federado y, sobre todo, de la tecnología Blockchain, así como revisión de trabajos previos que combinan ambas. Esta fase permitió identificar y entender las carencias del aprendizaje federado clásico en cuanto a trazabilidad y auditoría, y fundamentar la propuesta del proyecto.

2. **Diseño de la arquitectura**: definición de los dos bloques principales del sistema (la red Blockchain y la red de entrenamiento distribuido) y de la forma en que ambos se integrarían, estableciendo qué información se registraría on-chain y qué papel desempeñaría cada componente.

3. **Implementación de la red Blockchain**: despliegue de la red Hyperledger Besu mediante el Quorum Developer Quickstart y desarrollo del *smart contract* `FLRegistry`, encargado de registrar las contribuciones y los modelos agregados.

4. **Implementación del aprendizaje federado**: desarrollo del servidor y los clientes con el framework Flower, junto con el modelo de IA y la lógica de entrenamiento sobre el dataset de imágenes de tráfico.

5. **Integración de ambos sistemas**: conexión de la red de entrenamiento con la Blockchain y con IPFS, añadiendo la lógica de subida de modelos, registro de CID y verificación cruzada entre cliente y servidor.

6. **Experimentación y análisis**: diseño y ejecución de las distintas configuraciones experimentales, recogida de métricas y análisis de los resultados obtenidos.

7. **Redacción de la memoria**: documentación del trabajo realizado, llevada a cabo de forma secuencial a lo largo de todo el proyecto y consolidada en su fase final.

Aunque las fases se presentan de forma secuencial, en la práctica el trabajo ha tenido un carácter **iterativo**: los problemas detectados durante la integración o la experimentación obligaron en ocasiones a revisar decisiones de fases anteriores, como la estructura del *smart contract* o la adición de una métrica.

4.3 Stack tecnológico

A continuación se describen las principales tecnologías y herramientas empleadas, agrupadas por su función dentro del sistema.

4.3.1 Blockchain

- **Hyperledger Besu**: cliente de Ethereum empresarial utilizado como base de la red Blockchain permisionada, con consenso IBFT 2.0.

- **Tessera**: gestor de transacciones privadas asociado a cada nodo de la red.

- **Quorum Developer Quickstart**: herramienta de ConsenSys empleada para generar y desplegar la red de forma automatizada.

- **Solidity**: lenguaje en el que se ha desarrollado el *smart contract* `FLRegistry`.

- **Docker** y **Docker Compose**: para el mantenimiento de contenedores y la orquestación de los nodos de la red.

4.3.2 Aprendizaje federado e Inteligencia Artificial

- **Flower** (`flwr`): framework de aprendizaje federado que coordina el servidor y los clientes.

- **PyTorch**: librería empleada para la definición y el entrenamiento del modelo de red neuronal.
- **NumPy**: para la manipulación de los pesos del modelo.

4.3.3 Almacenamiento e integración

- **IPFS** (Kubo): sistema de almacenamiento distribuido para los modelos serializados.
- **web3.py**: librería para la interacción entre el código Python y la red Blockchain.
- **requests**: para la comunicación con la API de IPFS.

4.3.4 Lenguaje y entorno

- **Python 3.12**: lenguaje principal del lado del servidor y los clientes.
- **WSL2 (Ubuntu 22.04)**: entorno de ejecución sobre el que se han desplegado todos los componentes.
- **Git** y **GitHub**: para el control de versiones del código.

4.4 Estructura del proyecto

El código del proyecto se organiza en los siguientes directorios principales:

- `Scripts/`: contiene los scripts Python que implementan el servidor (`server.py`), el cliente (`client.py`), el modelo de IA (`model.py`) y el despliegue del contrato (`contract_address.py`).
- `Smart_Contracts/`: contiene el contrato Solidity `FLRegistry.sol`, su ABI compilado y los datos generados durante el despliegue (subdirectorio `Contract_Data/` con la dirección del contrato).
- `Dataset/`: contiene el dataset de imágenes de tráfico y el fichero CSV de etiquetado utilizado durante el entrenamiento.
- `Metricas/`: directorio donde se almacenan los archivos CSV con las métricas recogidas durante los experimentos, separados en `Client/`, `Server/` y `Pérdida/`.

Esta organización separa con claridad los tres grandes bloques del sistema (la lógica de entrenamiento, el componente Blockchain y los datos), lo que facilita el mantenimiento y una posible reproducibilidad posterior del proyecto.

La totalidad del código implementado se encuentra en el siguiente repositorio de GitHub: `https://github.com/dpz6-ua/Blockchain_ML`

5 Desarrollo

En este capítulo se explicará toda la investigación realizada del proyecto, incluyendo el proceso del diseño e implementación de la arquitectura de la red, los diferentes elementos que la componen, su despliegue y las variables a medir para obtener las conclusiones.

El sistema propuesto se basa en 2 bloques principales: la red de trazabilidad y auditabilidad de Blockchain y la red de entrenamiento del modelo de Inteligencia Artificial distribuido. Ambas redes se ejecutan por separado y no dependen la una de la otra, por lo que es necesario adaptarlas para poder aprovechar las características y ventajas que proporcionan cada una de ellas y unificarlas en un mismo sistema, permitiendo un entrenamiento más seguro y fiable. A continuación se explica la configuración y el despliegue de cada una de las redes en profundidad.

5.1 Red de Blockchain

La red Blockchain desarrollada en este trabajo representa el núcleo y una de las bases del proyecto. Es la encargada de proporcionar trazabilidad, integridad y auditoría durante el proceso de entrenamiento distribuido del modelo de Inteligencia Artificial. A diferencia de un sistema de aprendizaje federado tradicional, donde los intercambios de modelos se realizan de manera centralizada y sin un mecanismo robusto de auditoría, la incorporación de una red Blockchain permite mantener un registro inmutable de las contribuciones realizadas por cada participante durante las diferentes rondas de entrenamiento.

El objetivo principal de esta red es actuar como una capa de confianza adicional entre los participantes del sistema, permitiendo verificar qué cliente ha participado en cada ronda, qué modelo ha sido generado y cuál ha sido el resultado agregado final. De esta manera, se consigue un entorno de entrenamiento distribuido más seguro y transparente.

5.1.1 Arquitectura: ConsenSys y Hyperledger Besu

Teniendo claros los conceptos teóricos y los beneficios de este tipo de redes, es el momento de explicar la implementación real de esta red en este proyecto.

Dentro de las redes Blockchain empresariales existen dos plataformas compatibles con la Máquina Virtual de Ethereum (EVM) adoptadas y mantenidas por ConsenSys: **GoQuorum**, una distribución de *go-ethereum* escrito en Go, y **Hyperledger Besu**, mantenida por la Fundación Linux y escrita en Java. Ambas son compatibles con el uso de *smart contracts* y comparten herramientas del ecosistema Quorum (como Tessera para transacciones privadas), pero se diferencian en su implementación, comunidad y mecanismos de consenso disponibles.

Para acelerar el proceso de despliegue y configuración de la red, se ha utilizado el **Quorum Developer Quickstart**, una función por línea de comandos proporcionada por ConsenSys que permite generar una red Blockchain privada local en cuestión de minutos. La herramienta permite elegir la tipología de la red entre Besu y GoQuorum como cliente Ethereum, al igual que la opción de activar el soporte de transacciones privadas con Tessera. Para este proyecto se ha seleccionado **Hyperledger Besu** debido a su carácter abierto, su mantenimiento por parte de la Fundación Linux, su soporte nativo de los algoritmos de consenso más modernos (IBFT 2.0 y QBFT) y por la calidad de su documentación oficial.

La red generada por el quickstart por defecto está formada por 4 nodos validadores y un nodo RPC adicional (no validador) que actúa como punto de entrada para las aplicaciones externas, junto con un proxy EthSigner para la firma de transacciones. Esta topología sigue el requisito mínimo de los algoritmos BFT, según el cual el número de validadores n debe cumplir $n \geq 3f + 1$ para tolerar f nodos defectuosos, de manera que con 4 validadores la red soporta el fallo de uno de ellos.

La red desplegada finalmente en este proyecto utiliza IBFT 2.0 como algoritmo de consenso y Tessera como gestor de transacciones privadas, ambos descritos en el capítulo 2. Además, por motivos de experimentación y validación adicionales, se han añadido y configurado varios nodos más en la red, para realizar así una comparativa de rendimiento más profunda de lo que supone el aumento del tamaño de los nodos en una red de este estilo. Teniendo en cuenta en todo momento la problemática principal a la que se enfrenta el desarrollo de este proyecto, que es el entrenamiento distribuido en sistemas en ejecución en tiempo real.

5.1.2 Configuración de los Nodos

La red desplegada está formada por un conjunto de nodos validadores y un nodo RPC, cada uno de ellos ejecutado en un contenedor Docker independiente, lo que permite una mayor escalabilidad y reproducibilidad en el experimento. La configuración por defecto generada por el quickstart es la siguiente:

- **4 nodos validadores** (`validator1` a `validator4`): responsables de proponer y validar los bloques mediante el algoritmo de consenso.
- **1 nodo RPC** (`rpcnode`): nodo no validador que expone los puntos de acceso JSON-RPC y WebSocket hacia las aplicaciones cliente, en este caso los participantes del entrenamiento federado.
- **1 instancia de Tessera por nodo**: encargada de gestionar las transacciones privadas asociadas a cada nodo Besu.
- **1 contenedor EthSigner**: proxy de firma de transacciones que facilita la integración con aplicaciones externas.

Cada nodo tiene los siguientes archivos de configuración:

- **nodekey** y **nodekey.pub**: claves que identifican de manera única al nodo en la red P2P y permiten derivar su *enode URL*.

- **accountkey**: par de claves ECDSA (secp256k1) utilizado para firmar transacciones desde el nodo.
- **Claves de Tessera**: par de claves NaCl independientes para el cifrado de transacciones privadas.
- **genesis.json**: archivo compartido por todos los nodos que define la configuración inicial de la cadena (`chainID`, algoritmo de consenso, validadores iniciales, asignaciones de balance inicial, etc...).

Para añadir un nuevo nodo a la red, el procedimiento consiste en:

1. Generar las claves del nuevo nodo mediante herramientas de Besu (`besu public-key export` y `besu public-key export-address`).
2. Distribuir el archivo `genesis.json` y la lista de *bootnodes* actualizada al nuevo nodo.
3. Añadir el nuevo servicio al archivo `docker-compose.yml`, asignando una dirección IP libre dentro de la subred `quorum-dev-quickstart` y montando el directorio de claves correspondiente.
4. Arrancar el contenedor del nuevo nodo y verificar la sincronización con el resto de la red mediante `net_peerCount` y `eth_syncing`.

Siguiendo los pasos anteriores, en la red Blockchain de este proyecto se añadieron 3 nodos adicionales, aumentando a 7 el número de nodos validadores totales en la red en una parte de la experimentación.

5.1.3 Desarrollo del Smart Contract: FLRegistry

El smart contract `FLRegistry`, está escrito en Solidity (`0.8.10`) y representa el elemento lógico que conecta la red Blockchain con el sistema de aprendizaje federado. Su función es registrar de manera inmutable las contribuciones de cada cliente y de los modelos globales agregados a lo largo de las distintas rondas del entrenamiento. Se despliega una única vez sobre la red Besu antes del inicio del experimento (después de poner en marcha la red Blockchain), y todos los participantes (clientes y servidor agregador) interactúan con él a través de su dirección desplegada.

A continuación se muestra el código del contrato implementado y posteriormente se detallarán cada uno de los elementos que lo componen.

Código 5.1: Contrato FLRegistry

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;

contract FLRegistry {
    struct ModelRound {
        uint256 roundNum;
        string modelHash;
        address author;
        uint256 timestamp;
    }

    mapping(uint256 => ModelRound) public history;
    mapping(uint256 => mapping(address => ModelRound)) public clientHistory;

    uint256 public latestRound;
    bool public hasModel;

    event ModelUpdated(uint256 round, string modelHash, address author);

    function registerClientModel(uint256 round, string memory hash) public {
        clientHistory[round][msg.sender] = ModelRound(round, hash, msg.sender, ↩
            ↪ block.timestamp);
        emit ModelUpdated(round, hash, msg.sender);
    }

    function updateModel(uint256 round, string memory hash) public {
        history[round] = ModelRound(round, hash, msg.sender, block.timestamp);
        latestRound = round;
        hasModel = true;
        emit ModelUpdated(round, hash, msg.sender);
    }

    function getLastModel() public view returns (ModelRound memory) {
        require(hasModel, "No hay modelos registrados");
        return history[latestRound];
    }

    function getClientModel(uint256 round, address client) public view returns ↩
        ↪ (string memory) {
        return clientHistory[round][client].modelHash;
    }
}
```

Estructura de datos: ModelRound

La unidad básica de información del contrato es la estructura `ModelRound`, que representa una entrada del registro. Cada entrada contiene cuatro campos:

- `roundNum`: Representa el número de la ronda al que pertenece el registro. Actúa también

como clave para facilitar la lectura del registro completo en una consulta.

- `modelHash`: Hash del modelo serializado, almacenado como string para mantener una representación legible (en hexadecimal). Sirve de identificador del modelo enviado fuera de la red.

- `author`: Dirección del entorno de Ethereum que realizó el registro. Identifica al cliente o al agregador responsable de la transacción, siendo la Blockchain la encargada de autenticarlo con msg.sender.

- `timestamp`: Representa el instante de la transacción, tomado de `block.timestamp`. Permite trazar el orden temporal de la secuencia de transacciones registradas.

Almacenamiento: los dos mappings

El contrato tiene dos estructuras de almacenamiento que muestran los dos tipos de información que el sistema audita:

- `history`: Mapeo del número de ronda a la información del modelo generado en cada ronda. Almacena el modelo global agregado de cada ronda generado por el servidor tras combinar las contribuciones de los clientes. Representa el registro oficial del modelo final de cada iteración del entrenamiento federado.

- `clientHistory`: Mapeo del número de ronda y la información del modelo en cada ronda por cada uno de los clientes. Almacena las contribuciones individuales de cada cliente en cada ronda. Además, permite tener un registro de qué clientes participaron en una ronda determinada y qué versión del modelo aportaron.

Separar entre el registro global y el registro por cliente permite distinguir la trazabilidad de cada participante (`clientHistory`) de la trazabilidad del modelo resultante de la agregación (`history`).

Además, el contrato tiene dos variables auxiliares, que son el `latestRound`, que actúa como puntero a la última ronda completada, y el `hasModel`, un flag booleano que indica si el registro global contiene al menos una entrada. De esta manera se evita tener que recorrer los mappings, obteniendo una mayor eficiencia a la hora de acceder a los datos.

Funciones de escritura

El contrato tiene dos funciones de escritura, una para cada uno de los dos roles del sistema comentados en la sección anterior de los mapeos:

- `registerClientModel(round, hash)`: Es invocado por los clientes al terminar su entrenamiento local en una ronda, registrando en `clientHistory` su contribución en una determinada ronda. Se utiliza el `msg.sender`, garantizando que cada cliente solo pueda escribir bajo su propia dirección, estando autenticado por la red de Blockchain.

- `updateModel(round, hash)`: Es invocado por el servidor agregador después de combinar las contribuciones recibidas por parte de todos los cliente de la red y de producir un nuevo modelo global. Actualiza el registro de la ronda en `history` y avanza el puntero `latestRound`. La primera invocación además activa el flag `hasModel`.

Ambas funciones emiten un evento llamado `ModelUpdated`, lo que permite comunicar a los participantes de la red las diferentes modificaciones que se producen en el modelo a lo largo de la ejecución de las rondas.

Funciones de lectura

Las funciones de lectura `getLastModel()` y `getClientModel(round, client)` no modifican el estado y no consumen gas cuando se invocan desde fuera del contrato:

- `getLastModel()`: Utilizada por los clientes para obtener el CID (Identificador de contenido) del último modelo global registrado en la Blockchain por el servidor. Este CID se utiliza para descargar el modelo desde un servidor de almacenamiento distribuido, IPFS, y comparar sus pesos con los recibidos a través de la red de entrenamiento, verificando así que el modelo distribuido por el servidor coincide con el que ha sido publicado en la cadena.

- `getClientModel(round, client)`: Utilizada por el servidor para consultar la contribución de un cliente específico en una ronda concreta. Permite comprobar que el CID declarado por el cliente al enviar sus pesos coincide con el que ha registrado on-chain (dentro de la red Blockchain), descartando la contribución en caso contrario.

La función `getLastModel()` tiene una comprobación adicional del flag `hasModel` para evitar errores de acceso de elemento indefinido.

Aspectos adicionales del diseño

A la hora de diseñar el contrato se ha intentado ofrecer una eficiencia lo más óptima posible junto con una lógica funcional y simple. Algunos elementos que cabe destacar son los siguientes:

- Se registra el hash del modelo y no los pesos completos, ya que almacenar pesos on-chain sería demasiado caro y además expondría información sensible. El hash actúa como un identificador del modelo, lo que permite verificar después la veracidad del modelo intercambiado off-chain (durante el proceso de entrenamiento federado).

- La autenticación se delega en el propio protocolo: cualquier escritura queda firmada por la cuenta emisora, y `msg.sender` se utiliza como identificador del autor sin necesidad de mecanismos adicionales.

- No se ha incluido control de acceso (con listas de clientes autorizados por ejemplo) porque la red en la que se despliega el contrato es permisionada, es decir, el control de acceso ya se aplica a nivel de red, donde solo los nodos autorizados pueden conectarse y enviar transacciones.

5.1.4 Despliegue de la red Blockchain

El despliegue de esta red en este proyecto es muy automático ya que se realiza con un Docker Compose, lo cual permite levantar toda la infraestructura con un pequeño conjunto de comandos, permitiendo además una reproducibilidad de la experimentación muy alta. El despliegue se divide en 3 fases: instalación y generación de los artefactos de la red, arranque de los contenedores y despliegue del smart contract.

Fase 1: Generación de los artefactos de la red

El primer paso consiste en generar los archivos de configuración necesarios para la red Besu: el archivo `genesis.json`, las claves de los nodos y los archivos `docker-compose.yml` correspondientes. Se necesitará tener instalado `Node.js`, `Docker Compose` y `cURL`. Esta tarea se realiza ejecutando el siguiente comando:

Código 5.2: Generación de los artefactos del quickstart

```
npx quorum-dev-quickstart
```

Cuando se ejecute este comando, entraremos en un proceso guiado de instalación, donde deberemos especificar una serie de parámetros, como es el cliente utilizado (Hyperledger Besu), la activación del soporte de transacciones privadas mediante Tessera y habilitar el stack de monitorización por defecto (Prometheus y Grafana). El resultado es un directorio `quorum-test-network` que contiene toda la configuración necesaria para arrancar una red de 4 nodos validadores, 1 nodo RPC y sus correspondientes instancias de Tessera, tal y como se ha especificado en la definición de la red anteriormente.

Fase 2: Arranque de la red

Una vez generados los artefactos, la red se levanta con el script `run.sh` incluido en el directorio del quickstart, que realmente lo que hace es invocar a `docker compose up -d`, construyendo las imágenes necesarias y arrancando todos los contenedores de los nodos especificados en el `docker-compose.yml`:

Código 5.3: Arranque de la red Besu

```
cd quorum-test-network
./run.sh
```

Tras el arranque, se puede verificar que la red está operativa y que los nodos se han descubierto entre sí. Esto se comprueba mediante una llamada JSON-RPC al nodo RPC, consultando el número de *peers* conectados:

Código 5.4: Verificación de la sincronización

```
curl -X POST --data \
  '{"jsonrpc":"2.0","method":"net_peerCount","params":[],"id":1}' \
  -H "Content-Type: application/json" http://localhost:18545
```

Una respuesta con un valor hexadecimal igual o superior a `0x4` indica que el nodo RPC ha establecido conexión con los cuatro validadores 5.1. Adicionalmente, el quickstart expone

una interfaz web local (Quorum Explorer) en `http://localhost:25000` con la cual se puede ver el estado de la red, los bloques minados y las transacciones procesadas 5.2.

```
david@David:~/tfg/quorum-test-network$ curl -X POST --data   '{"jsonrpc":"2.0","method":"net_peerCount"
,"params":[],"id":1}'   -H "Content-Type: application/json" http://localhost:8545
{
  "jsonrpc" : "2.0",
  "id" : 1,
  "result" : "0x7"
```

Figura 5.1: Conexión establecida con la red Blockchain desplegada

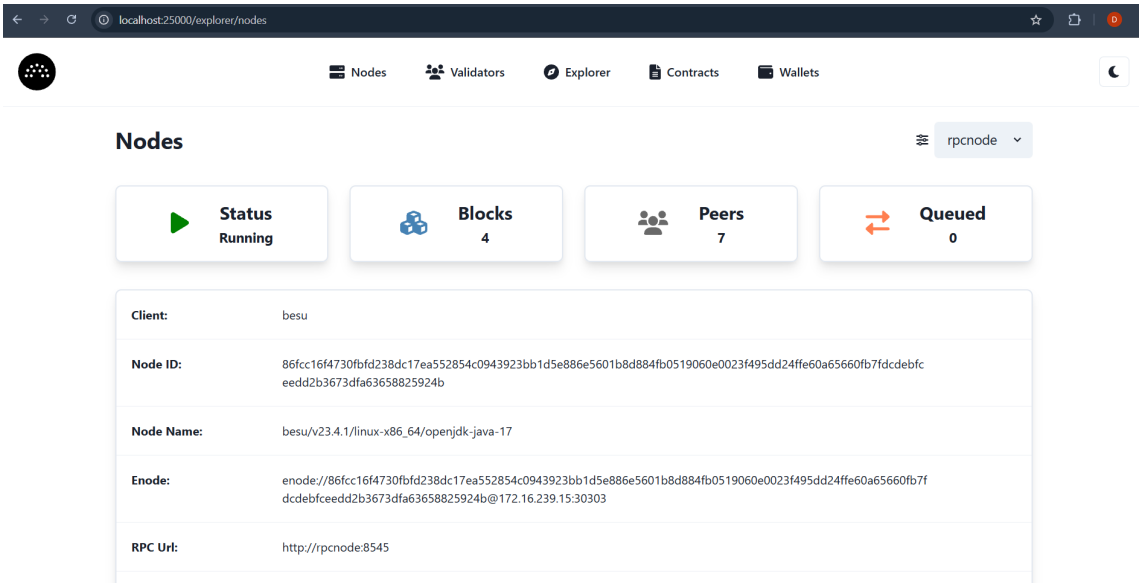


Figura 5.2: Interfaz web Quorum Explorer

Fase 3: Despliegue del Smart Contract

Con la red ya operativa, el siguiente paso es desplegar el contrato `FLRegistry` sobre la cadena. Este despliegue se realiza de manera previa al entrenamiento federado, y se hace mediante un script en Python (`contract_address.py`) que utiliza la librería `web3.py`. El script realiza las siguientes operaciones:

1. Conexión al nodo RPC del quickstart con un proveedor HTTP apuntando a `http://localhost:18545`.
2. Compilación del contrato Solidity generando el bytecode y el ABI.
3. Construcción y firma de la transacción de despliegue con la cuenta del administrador.
4. Envío de la transacción y espera de la confirmación de minado.
5. Almacenamiento de la dirección del contrato y el ABI en un archivo JSON compartido (`FLRegistry_info.json`), accesible tanto por el servidor agregador como por los clientes Flower.

A continuación se podrán observar los fragmentos del código del `contract_address.py`, encargado del despliegue del contrato en la red Blockchain, siguiendo los pasos mencionados en el párrafo anterior.

Código 5.5: Script de despliegue del contrato (fragmentado)

```
***

# Establecimiento de la conexión

ETH_ENDPOINT = 'http://localhost:18545'
w3 = Web3(Web3.HTTPProvider(ETH_ENDPOINT))

***

# Compilación del contrato y generación de bytecode y ABI

flreg_path = "../Smart_Contracts/FLRegistry.sol"

install_solc("0.8.10")
with open(flreg_path, "r") as file:
    contract_source_code = file.read()

compiled_sol = compile_standard({
    "language": "Solidity",
    "sources": {flreg_path: {"content": contract_source_code}},
    "settings": {"outputSelection": {"*": {"*": ["abi", "evm.bytecode", "
        ↪ metadata"]}}},
}, solc_version="0.8.10")

bytecode = compiled_sol["contracts"][flreg_path]["FLRegistry"]["evm"]["
    ↪ bytecode"]["object"]
abi = compiled_sol["contracts"][flreg_path]["FLRegistry"]["abi"]

***

# Envío de transacción de despliegue de contrato

FLRegistry = w3.eth.contract(abi=abi, bytecode=bytecode)
transaction = FLRegistry.constructor().build_transaction({
    'from': member1_address,
    'nonce': w3.eth.get_transaction_count(member1_address),
    'gas': 2000000,
    'gasPrice': w3.eth.gas_price,
    'chainId': w3.eth.chain_id
})

signed_tx = w3.eth.account.sign_transaction(transaction, priv_key1)
tx_hash = w3.eth.send_raw_transaction(signed_tx.raw_transaction)
print(f"Transacción enviada: {tx_hash.hex()}. Esperando confirmación")

tx_receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
```

```
45
46***
47
48# Guardado del contrato en un directorio compartido
49
50with open("../Smart_Contracts/Contract_Data/FLRegistry_info.json", "w") as f:
51    json.dump({"address": tx_receipt.contractAddress, "abi": abi}, f)
52print("\nInformación guardada en '../Smart_Contracts/Contract_Data/↩
    ↪ FLRegistry_info.json'")
```

Una vez ejecutado el script completo estando la red Blockchain levantada y operativa, podremos ver el siguiente conjunto de mensajes, mostrándose la dirección del nodo que lanza la transacción, el hash de la transacción y, finalmente, cuando el despliegue se haya minado en la red, veremos un mensaje de confirmación del despliegue junto con la dirección asociada al contrato en la red, guardada además en el archivo correspondiente 5.3.

```
david@David:~/tfg/Federated_Learning/Scripts$ python3 contract_address.py
Despliegue desde nodo: 0xFE3B557E8Fb62b89F4916B721be55cEb828dBd73
Transacción enviada: 8d399acae21373519a93965e86a54cb7b10aadf00fa546d0044277e4e185fb3f.
Esperando confirmación

========================================
Contrato desplegado
Dirección: 0x42699A7612A82f1d9C36148af9C77354759b210b
========================================

Información guardada en '../Smart_Contracts/Contract_Data/FLRegistry_info.json'
```

Figura 5.3: Despliegue del contrato en la red Blockchain

Resultado final

Tras completar las tres fases, el sistema queda en el siguiente estado:

- Una red Hyperledger Besu con 4 validadores (o 7 dependiendo del experimento) y 1 nodo RPC operativa, con consenso IBFT 2.0 y soporte de transacciones privadas vía Tessera.
- El contrato `FLRegistry` desplegado en la cadena, con su dirección y ABI almacenados en un archivo de configuración compartido.
- Las herramientas de monitorización web (Prometheus y Grafana) accesibles para inspeccionar el comportamiento de la red durante los experimentos.

A partir de este punto, la red está lista para recibir las transacciones generadas por los clientes y el servidor agregador del sistema de aprendizaje federado, cuya implementación se describe en la siguiente sección.

5.2 Red de entrenamiento distribuido

La segunda pieza principal del sistema es la red de entrenamiento distribuido, encargada de coordinar el aprendizaje distribuido de un modelo de Inteligencia Artificial entre varios clientes sin necesidad de centralizar los datos. Para ello se ha utilizado **Flower** (`flwr`), un framework "open source" especializado en aprendizaje federado que ofrece compatibilidad con las principales librerías de Deep Learning (en este proyecto, PyTorch) y un modelo de programación simple basado en la separación entre cliente y servidor.

5.2.1 Arquitectura

La red sigue una arquitectura **cliente-servidor** clásica del aprendizaje federado, complementada con un sistema de almacenamiento distribuido. Concretamente, se compone de los siguientes elementos:

- Un servidor agregador (`server.py`), que implementa una estrategia personalizada `MiBesuServer` extendiendo `fl.server.strategy.FedAvg`, el cual coordina las rondas de entrenamiento, distribuye el modelo global, recibe las actualizaciones de los clientes y las combina finalmente mediante FedAvg.
- Un conjunto de clientes (`client.py`), implementados como subclases de `fl.client.NumPyClient`. En la configuración creada para los experimentos participan dos clientes, identificados como *member 2* y *member 3*.
- Un servidor IPFS (*Kubo*) local que actúa como almacenamiento distribuido para los modelos serializados. Los pesos no se transmiten directamente a través de la Blockchain, sino que se suben a IPFS y se intercambia únicamente el identificador de contenido (CID) resultante.

Combinar Flower, IPFS y Blockchain resuelve uno de los principales inconvenientes de almacenar modelos completos en una cadena: el coste excesivo y prohibitivo de almacenamiento on-chain. Los modelos quedan disponibles en el servidor IPFS (que son recuperables por cualquier participante con acceso a la red) y la Blockchain almacena únicamente sus CID, que son referencias inmutables y verificables del contenido.

Otro aspecto importante de la arquitectura es que cada cliente se conecta a un nodo RPC distinto de la red Besu (puertos `20002` y `20004` para los *members* 2 y 3 respectivamente), mientras que el servidor lo hace al nodo RPC principal del quickstart (puerto `18545`). Con esto distribuye la carga sobre la red Blockchain en lugar de concentrarla en un único punto de entrada, y permite además ejecutar los clientes en máquinas físicamente distintas, escenario que se contempla y se explicará en uno de los experimentos realizados en los siguientes apartados de esta memoria.

5.2.2 Modelo de IA usado

El caso de uso elegido para los experimentos es un problema de clasificación de imágenes de tráfico en cuatro categorías: `accident`, `dense_traffic`, `meteorology` y `sparse_traffic`.

El conjunto de datos utilizado procede del repositorio FL-Blockchain del grupo CloudLab-AIA (CloudLab-AIA, 2026), y se ha reutilizado en este proyecto para mantener la línea de investigación del grupo. El dataset, además, está organizado en un fichero CSV en el que cada fila contiene la ruta de una imagen y su etiqueta asociada a una de las 4 clases mencionadas anteriormente.

El modelo está definido en la clase `NetCliente` (módulo `model.py`) y es una red neuronal convolucional sencilla, formada por dos bloques convolucionales (con max pooling entre ellos) seguidos de dos capas completamente conectadas (fully connected), con salida de 4 neuronas correspondientes a las clases del problema. Las imágenes se redimensionan a 32×32 píxeles y se normalizan antes de entrar a la red.

El entrenamiento se realiza con la función de pérdida `CrossEntropyLoss` y el optimizador Adam ($\text{lr} = 0{,}001$), durante 3 épocas locales por ronda y un total de 30 rondas federadas. La partición de datos entre clientes es IID, (Independent and Identically Distributed) lo que significa que cada cliente recibe una porción aleatoria del dataset con una distribución de clases equivalente a la del conjunto completo. Es decir, cada cliente tiene la misma proporción de imágenes por cada categoría que los demás clientes.

Cabe destacar que la elección del modelo no es el foco principal de este trabajo, es decir, cualquier otra arquitectura de clasificación de imágenes encajaría sin modificaciones en el sistema, ya que la integración con Blockchain e IPFS es independiente de la red neuronal utilizada. El modelo actúa, en este contexto, como un caso de prueba sobre el que medir las propiedades del sistema de entrenamiento federado.

5.2.3 Proceso de entrenamiento federado

El entrenamiento se organiza en rondas sucesivas coordinadas por el servidor de Flower. Cada ronda sigue siempre la misma secuencia:

1. El servidor envía a cada cliente los parámetros del modelo global actual junto con la configuración de la ronda (incluido el número de ronda `server_round` mediante la callback `fit_config`).

2. Cada cliente realiza una serie de comprobaciones previas (descritas en la siguiente sección de integración) y, si el modelo recibido es válido, entrena durante 3 *epochs* sobre su partición local.

3. El cliente sube los pesos resultantes a IPFS, registra el CID en la Blockchain y los devuelve al servidor junto con el CID y su dirección Ethereum, que el servidor utilizará para verificar la contribución.

4. El servidor verifica cada contribución contra la Blockchain antes de agregar (sección de integración) y aplica la estrategia **FedAvg** sobre las contribuciones válidas, que se describe a continuación:

$$w_{global}^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \, w_k^{(t)}$$

donde $w_k^{(t)}$ son los pesos del cliente k en la ronda t, n_k su número de muestras y $n = \sum_k n_k$ el total.

5. El servidor sube el modelo global agregado a IPFS y registra su CID en Blockchain, dejando la ronda completamente registrada.

6. Se inicia la siguiente ronda.

En la siguiente figura 5.4 se puede ver la secuencia que sigue el entrenamiento de un cliente, donde éste recibe el modelo del servidor y, si después de corroborarlo con la Blockchain es correcto, lanza un mensaje de éxito, inicia el entrenamiento y registra el CID del modelo entrenado en Blockchain. Posteriormente se evalúa el modelo, se muestra la precisión que ha obtenido y se envían los pesos de éste a través de la red de entrenamiento federado.

```
INFO :      Sent reply
INFO :
INFO :      Received: evaluate message b7a269ad-67e6-49ea-9aea-12b1277fc29c
[3] Evaluando modelo
[3] Accuracy: 0.8829
INFO :      Sent reply
INFO :
INFO :      Received: train message 5adad1f7-a1df-4e01-9078-29541755beef
[3] Modelo del servidor correcto
[3] Iniciando entrenamiento local
0
1
2
[3] Hash registrado en Blockchain: QmZjjbynkjgVNZ5tP2XcxgWauuDo1jDrXSZUQn9JWubLG5
INFO :      Sent reply
INFO :
INFO :      Received: evaluate message 5dbbba6d-0ee8-4855-afdb-2898efbac1d9
[3] Evaluando modelo
[3] Accuracy: 0.9370
INFO :      Sent reply
INFO :
INFO :      Received: reconnect message e078f8b5-1957-4e5a-9c1d-21ef5d20c07c
INFO :      Disconnect and shut down
```

Figura 5.4: Ejemplo de ejecución del entrenamiento de un cliente

Adicionalmente, el servidor está configurado con `min_fit_clients=2` y `min_available_clients=2`, lo que significa que no se inicia una ronda hasta que ambos clientes están disponibles, y se necesita la participación de los dos para avanzar.

Cuando termina el entrenamiento, en el momento que se ha completado la totalidad de las rondas configuradas, se muestra un resumen (*summary*) de la evolución de la función de pérdida a lo largo de las rondas, tal y como se muestra en la siguiente figura 5.5

```
INFO :      configure_evaluate: strategy sampled 2 clients (out of 2)
INFO :      aggregate_evaluate: received 2 results and 0 failures
INFO :
INFO :      [SUMMARY]
INFO :      Run finished 3 round(s) in 376.45s
INFO :          History (loss, distributed):
INFO :                  round 1: 0.5790393352508545
INFO :                  round 2: 0.31198613345623016
INFO :                  round 3: 0.20242171734571457
INFO :
```

Figura 5.5: Final del entrenamiento

5.2.4 Comunicación cliente-servidor

La comunicación entre clientes y servidor Flower se realiza mediante **gRPC**, sobre el puerto `8081` en este proyecto. Existen dos interfaces principales proporcionadas por Flower:

- En el lado del cliente, la clase creada para este proyecto `MiFlowerClient` hereda de `fl.client.NumPyClient`. Esta clase implementa los métodos `get_parameters()`, `set_parameters()`, `fit()` y `evaluate()`, en los que se ha integrado además la lógica de IPFS y Blockchain.
- En el lado del servidor, la clase `fl.server.strategy.FedAvg`, de la que hereda `MiBesuServer`. En esta clase se ha sobreescrito el método `aggregate_fit()` para introducir la verificación contra Blockchain de los clientes antes de la agregación.

Además, el servidor inyecta en cada ronda el número de ronda actual a través de la *callback* `on_fit_config_fn`, lo que permite a los clientes saber con qué identificador deben registrar su contribución en el contrato.

5.3 Integración IA + Blockchain

La integración entre el aprendizaje federado y la Blockchain es un punto clave de los objetivos de este trabajo. Ambos sistemas funcionan de manera independiente, pero el código de los clientes y del servidor se ha adaptado para realizar las operaciones de IPFS y Blockchain en los momentos principales de la ejecución del sistema. El resultado es un sistema de verificación bidireccional, el cual protege tanto al servidor frente a clientes maliciosos como a los clientes frente a un servidor malicioso.

5.3.1 Conexión de ambos sistemas

La conexión se ha implementado en Python utilizando tres librerías principales:

- `web3.py`: Sirve para interactuar con el nodo RPC de la red Besu mediante JSON-RPC. Permite construir, firmar y enviar transacciones al contrato `FLRegistry`, al igual que permite realizar llamadas de lectura.
- `requests`: Sirve para comunicarse con el nodo IPFS local. Se utiliza la API HTTP de Kubo (puerto `5001`) para subir contenido y el gateway HTTP (puerto `8080`) para descargarlo a partir de su CID.
- `pickle`: Sirve para serializar los pesos del modelo (en forma de lista de arrays NumPy) antes de subirlos a IPFS. Se usa además para deserializarlos en el momento de la descarga.

La dirección y el ABI del contrato se obtienen del archivo `FLRegistry_info.json`, generado durante el despliegue por el script `contract_address.py`, como se ha mencionado en las secciones previas. Cada participante dispone de una cuenta Ethereum propia con su clave privada:

- El servidor utiliza la cuenta del *member 1* del quickstart, conectándose al nodo RPC principal en `http://{SERVER_IP}:18545`.
- Cada cliente utiliza la cuenta correspondiente a su argumento `--member`, conectándose a un nodo distinto: `http://{CLIENT_IP}:20002` para el cliente 2 y `http://{CLIENT_IP}:20004` para el cliente 3.

En esta implementación las claves privadas están definidas directamente en el código fuente como un diccionario `CLIENT_ACCOUNTS` (hardcoded), principalmente para simplificar el entorno de desarrollo y la experimentación. En un despliegue de producción real, estas claves deberían gestionarse mediante un *keystore* cifrado o un servicio de gestión de secretos (utilizando herramientas como la gestión de claves (Key Vault) de Microsoft Azure, por ejemplo), tal como se contempla en la sección de trabajo futuro.

5.3.2 Flujo completo: preparación, entrenamiento, agregación y registro

El flujo completo de una ronda integrando los tres componentes (Flower, IPFS y Blockchain) se desarrolla del siguiente modo:

5.3.2.1 Preparación inicial (una sola vez)

1. Se levanta la red Besu mediante `./run.sh`.
2. Se ejecuta `contract_address.py`, que compila y despliega el contrato `FLRegistry` y guarda la información resultante en `FLRegistry_info.json`.
3. Se arranca el servicio de IPFS con `ipfs daemon`.
4. Se arranca el servidor con `python server.py`. Como parte de su inicialización, el servidor:
   - Instancia un modelo `NetCliente` con pesos aleatorios y lo serializa.
   - Sube esos pesos iniciales a IPFS, obteniendo un CID.
   - Registra ese CID en la Blockchain como modelo de la "ronda 0" mediante `updateModel(0, CID)`. Esto garantiza que cuando los clientes se conecten ya exista un modelo global de referencia en la cadena.
5. Se arrancan los clientes con `python client.py --member <ID>`. Cada cliente se conecta al servidor Flower vía gRPC y queda a la espera de la primera ronda, en el momento que hay suficientes clientes conectados en la red.

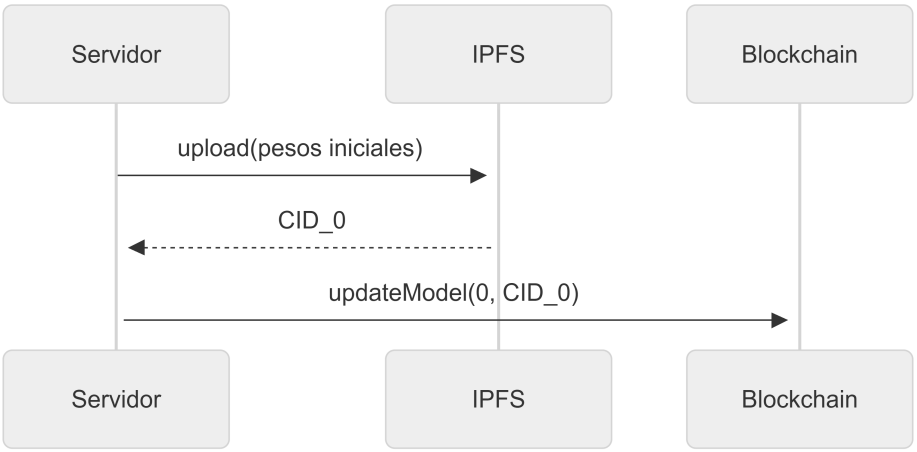


Figura 5.6: Preparación inicial del sistema

5.3.2.2 Durante cada ronda

1. El servidor distribuye los pesos del modelo global actual a los clientes.
2. **Verificación previa en el cliente**: antes de entrenar, el cliente descarga del CID registrado on-chain el modelo global y lo compara con el que ha recibido del servidor (función `check_model_CID`). La comparación se realiza de manera iterativa sobre cada tensor de pesos. Si no coinciden, el cliente **aborta el entrenamiento** y devuelve un error, evitando así contribuir sobre un modelo que puede ser manipulado.

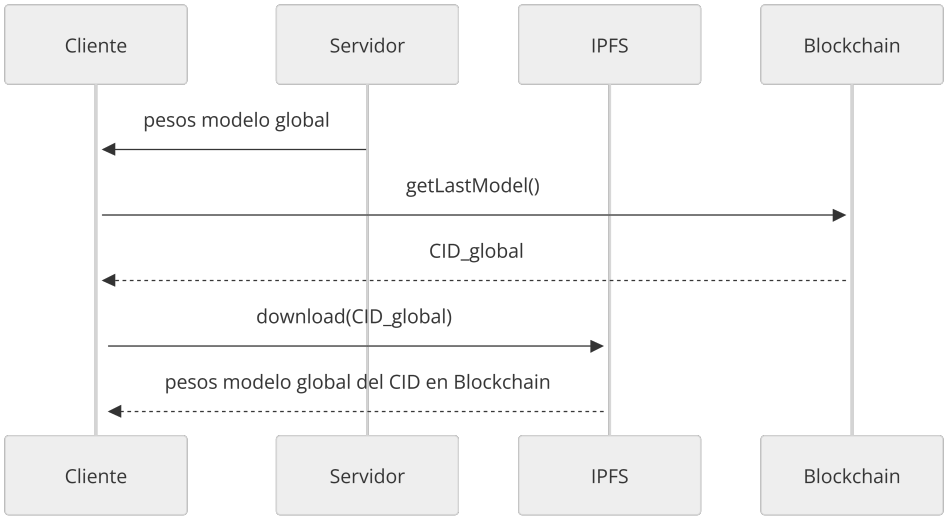


Figura 5.7: Inicio de ronda y verificación de los pesos iniciales por parte del cliente

3. Si la verificación es correcta, el cliente entrena el modelo durante 3 épocas sobre su partición local.
4. El cliente serializa los nuevos pesos con `pickle` y los sube a IPFS mediante el endpoint `/api/v0/add` de Kubo, obteniendo el CID resultante.
5. El cliente invoca `registerClientModel(round, CID)` sobre el smart contract definido en las secciones anteriores (FLRegistry), firmando la transacción con su clave privada. Durante esta operación se mide el tiempo de confirmación de la transacción y el gas consumido, que se guardan en el archivo de métricas del cliente. El apartado de las medidas se explica en las secciones posteriores.
6. El cliente devuelve a Flower los pesos, junto con un diccionario de métricas que incluye el CID subido y su dirección Ethereum.

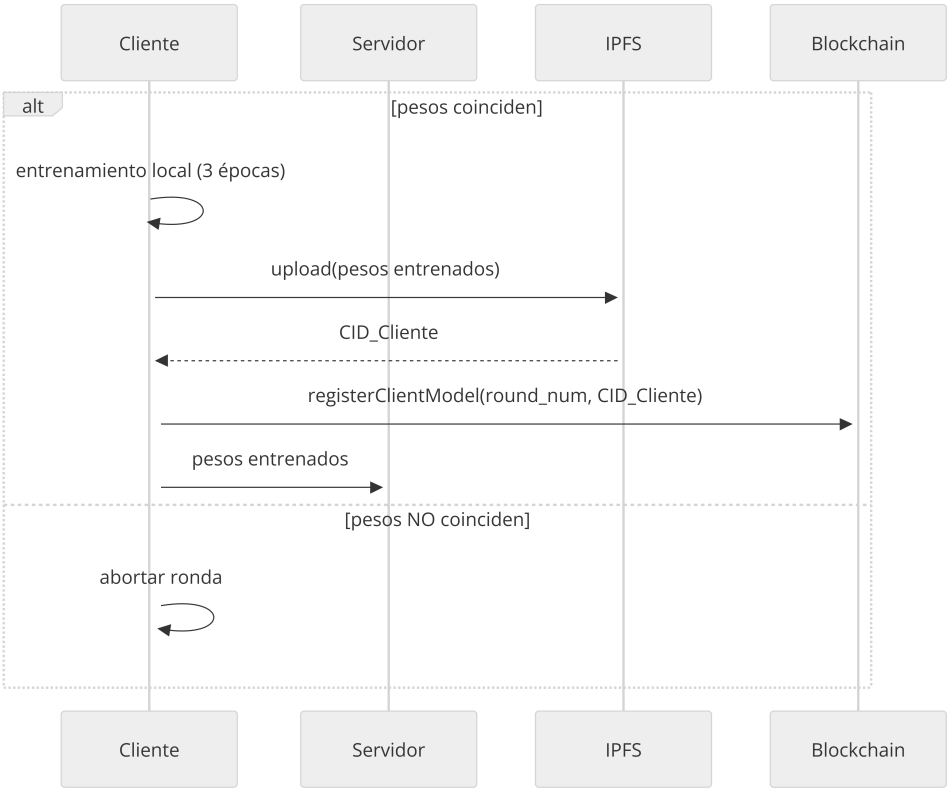


Figura 5.8: Entrenamiento y subida de pesos del cliente

7. **Verificación previa en el servidor**: en el método `aggregate_fit`, antes de agregar, el servidor consulta para cada cliente conectado la función `getClientModel(round, address)` del contrato y comprueba que el CID declarado por el cliente coincide con el registrado on-chain. Solo las contribuciones que pasan esta verificación se incluyen en la agregación.

8. El servidor aplica FedAvg sobre las contribuciones válidas.

9. El servidor serializa el modelo global resultante, lo sube a IPFS y registra el nuevo CID mediante `updateModel(round, CID)`.

10. Se inicia la siguiente ronda.

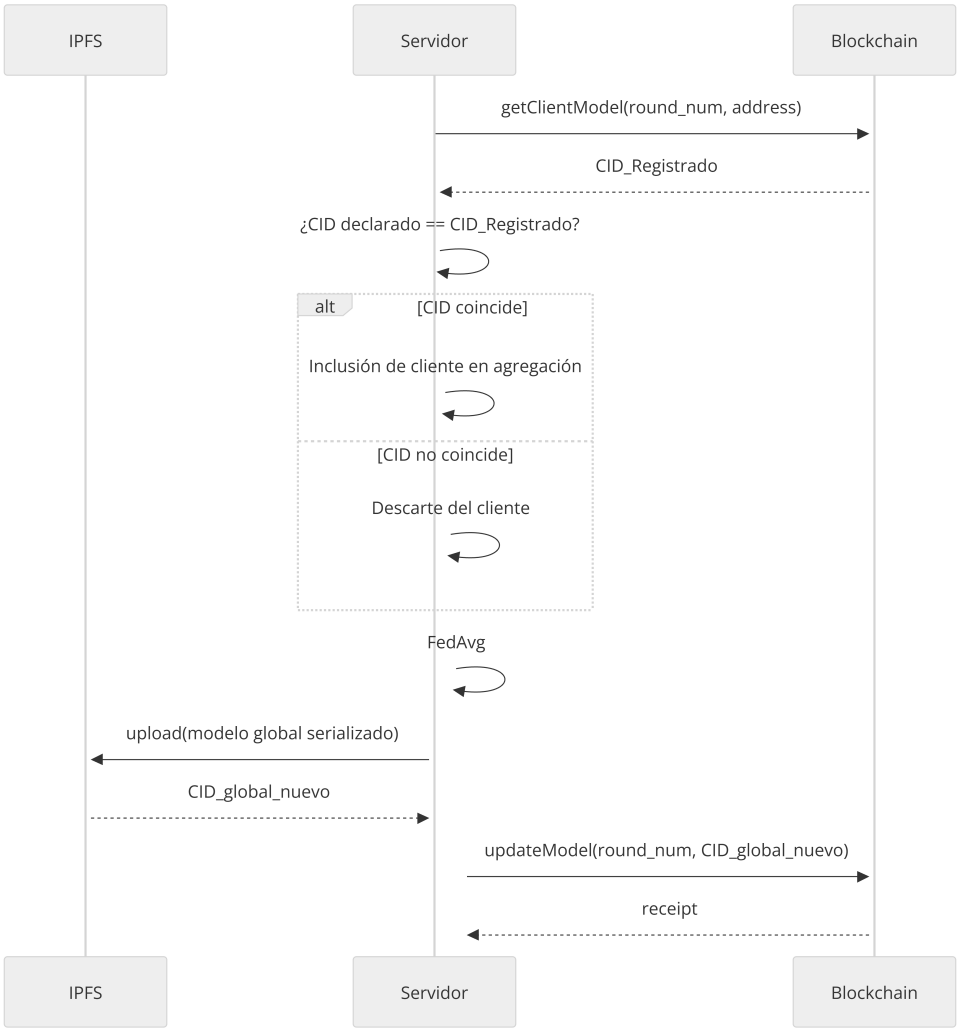


Figura 5.9: Verificación por parte del servidor y fin de ronda

5.3.3 Trazabilidad y garantías

La integración descrita proporciona una serie de garantías que no existen en una implementación de entrenamiento federado común. La aportación principal es la **doble verificación cruzada** entre cliente y servidor a través de la Blockchain, que permite que ninguno de los dos roles pueda actuar maliciosamente sin que el otro lo detecte, lo que permite las siguientes garantías:

- **Protección del cliente frente a un servidor malicioso**. Antes de invertir recursos en el entrenamiento, el cliente verifica que el modelo global que ha recibido coincide con el último CID registrado en la Blockchain por el servidor. Si el servidor intentase distribuir un modelo distinto del que ha publicado on-chain (por ejemplo, un modelo manipulado para un subconjunto de clientes), el cliente lo detectaría y abortaría la ronda.

- **Protección del servidor frente a clientes maliciosos**. Antes de incluir una contribución en la agregación, el servidor verifica que el CID declarado por el cliente coincide con el registrado on-chain bajo su dirección Ethereum para esa ronda. Como las transacciones de registro están firmadas por el cliente, este no puede modificar el registro una vez confirmado.
- **Integridad del contenido**. Los CID de IPFS son hashes criptográficos del propio contenido (*content-addressed storage*). Cualquier alteración de un modelo, por mínima que sea, cambia su CID, lo que haría que dejara de coincidir con el registrado on-chain y la verificación fallaría.
- **Trazabilidad completa del entrenamiento**. Al final del proceso, la Blockchain contiene un historial completo e inmutable de qué cliente participó en cada ronda, qué CID aportó y qué CID corresponde al modelo global de cada agregación. A partir de cualquier CID se puede recuperar el modelo exacto desde IPFS, lo que permite auditorías independientes del entrenamiento, incluso mucho tiempo después de haberlo realizado.
- **No repudio**. Las transacciones están firmadas con la clave privada del participante, por lo que ninguno puede negar haber realizado una contribución concreta.

Estas garantías no eliminan todas las posibles fuentes de error o ataque ya que un cliente malicioso podría seguir enviando un modelo mal entrenado aposta y registrar su CID correctamente. Del mismo modo, un servidor podría descartar contribuciones legítimas. Lo que sí permiten es elevar notablemente el nivel de confianza del sistema. Las posibles defensas adicionales contra ataques de envenenamiento y la incorporación de mecanismos de reputación basados en los registros on-chain se proponen como trabajo futuro.

Las métricas recogidas durante este proceso (gas consumido por transacción, tiempo de confirmación, tiempo total por ronda) se almacenan en archivos CSV diferenciados por cliente y por servidor, y constituyen la base de la experimentación y los resultados que se presentan en el capítulo siguiente.

5.4 Métricas recogidas

En esta sección se describirán las métricas que han sido utilizadas para poder evaluar el comportamiento del sistema. La recopilación de la información de estas se hace de manera automática en los momentos clave del flujo del sistema, tanto por la parte del cliente como del servidor. Las métricas se almacenan en archivos CSV separados por rol (cliente o servidor) y configuración del experimento (Sin Blockchain, con 4 o 7 validadores...), dentro del directorio `Metricas/`, del repositorio del proyecto, lo que permite un análisis posterior con herramientas como Pandas o Matplotlib. A continuación se explicará qué medidas recoge cada rol del sistema.

5.4.1 Métricas del cliente

Cada cliente registra, al final de cada ronda en un CSV propio los siguientes campos:

- **ronda**: número de ronda del entrenamiento federado.
- **gas_real**: cantidad de *gas* real consumido por la transacción `registerClientModel`, obtenido según el *receipt* devuelto por la Blockchain.
- **tiempo_confirmacion**: tiempo transcurrido entre el envío de la transacción y la recepción del *receipt* confirmando su inclusión en un bloque de la red.
- **tiempo_total_con_commit**: tiempo total del método `fit()`, incluyendo el entrenamiento local, la subida al servidor IPFS y la transacción a Blockchain. Permite medir el impacto que añade la capa de trazabilidad sobre el tiempo total del entrenamiento.
- **timestamp**: marca de tiempo.

5.4.2 Métricas del servidor

El servidor registra, al final de cada ronda en un CSV propio los siguientes campos:

- **ronda**: número de ronda del entrenamiento federado.
- **gas_consumido**: *gas* real consumido por la transacción `updateModel` al registrar el modelo global agregado en Blockchain.
- **tiempo_confirmacion_seg**: tiempo de confirmación de la transacción del servidor.
- **tiempo_total_ronda_seg**: tiempo total de la ronda desde que llega la primera contribución hasta que se publica el nuevo modelo global on-chain. Es la métrica principal para evaluar el rendimiento global del sistema.
- **timestamp**: marca de tiempo.

5.4.3 Métricas sin Blockchain

En una parte de la experimentación se ha ejecutado la red de entrenamiento distribuido sin la utilización de Blockchain. Esto permite comprobar el ”overhead” real de la ejecución del mismo sistema usando Blockchain para autenticación ni trazabilidad. Las métricas recopiladas en este caso difieren entre las comentadas anteriormente, tanto para servidor como para el cliente.

- **Servidor**: Registra el identificador de la ronda (`ronda`), el tiempo de agregación (`tiempo_agregacion_seg`) y el timestamp.
- **Clientes**: Registra el identificador de la ronda (`ronda`), el tiempo de entrenamiento completo con envío de datos al servidor (`tiempo_total_local`) y el timestamp.

5.4.4 Justificación de las métricas elegidas

Las métricas se han elegido para responder a las preguntas centrales del trabajo:

- **¿Cuánto cuesta integrar Blockchain en un sistema de aprendizaje federado?** Las métricas de *gas* permiten ver el coste en recursos de la red Blockchain por cada interacción.
- **¿Cómo escala el sistema con el número de validadores o de máquinas?** Los archivos CSV donde se depositan los resultados de las medidas inlcuyen sufijos como `_7vals` o `_2maquinas` y permiten comparar configuraciones distintas manteniendo el resto de variables constantes.
- Y probablemente la más importante, **¿cuánta latencia añade la Blockchain al proceso de entrenamiento?** Para ello se utilizan las métricas de tiempo de confirmación y tiempo total, las cuales permiten medir el coste adicional temporal frente a un sistema federado sin Blockchain.

A partir de estos datos, el capítulo siguiente analiza los resultados obtenidos en distintos escenarios experimentales.

6 Resultados

En este capítulo se muestran los experimentos realizados para evaluar el sistema desarrollado y el análisis de los resultados obtenidos. El objetivo es obtener las conclusiones necesarias para poder comprobar si esta red es asumible en un entorno real, midiendo el impacto que introduce la capa de seguridad (trazabilidad y autenticidad) de Blockchain sobre el sistema de aprendizaje federado. En concreto se pretende responder a las 3 preguntas formuladas en el desarrollo: el coste de integración de Blockchain y aprendizaje federado, el comportamiento del sistema con más validadores o máquinas y cuánto aumenta la latencia del sistema respecto a uno tradicional.

Para ello se han ejecutado una serie de experimentos que combinan tres factores importantes, que son el número de máquinas, la presencia o la ausencia de la capa Blockchain y el número de validadores de la red Besu. El análisis se realiza con las métricas obtenidas en tiempo de ejecución, descritas en el capítulo anterior.

6.1 Entorno de pruebas

Los experimentos se han ejecutado sobre dos máquinas conectadas a una misma red local. Las siguientes tablas resumen las características de hardware y software de cada una.

Componente	**Máquina 1**
Procesador	Intel Core i5-1135G7 @ 2.40 GHz
Núcleos / hilos	4 núcleos físicos, 8 hilos
Sistema operativo	Windows 11 + WSL2 (Ubuntu 22.04)
Recursos disponibles en WSL	8 hilos, 7,6 GB RAM

Tabla 6.1: Características de la Máquina 1

Componente	**Máquina 2**
Procesador	Intel Core i7-9700 @ 3.00 GHz
Núcleos / hilos	8 núcleos físicos, 8 hilos
Sistema operativo	Windows 11
Recursos disponibles	8 hilos, 8 GB RAM

Tabla 6.2: Características de la Máquina 2

El software utilizado en ambas máquinas es el siguiente, el cual ya ha sido definido y comentado en los apartados anteriores:

- **Hyperledger Besu** con Tessera, desplegado mediante el Quorum Developer Quickstart.
- **Flower** (`flwr`) como framework de aprendizaje federado.
- **IPFS** (Kubo `v0.32.1`) para el almacenamiento distribuido de los modelos.
- **Python 3.12**, con las librerías `web3`, `torch`, `requests` y `flwr`.

6.2 Diseño experimental

Para evaluar el sistema se han diseñado cinco configuraciones experimentales que combinan tres variables que son independientes entre sí. Son las siguientes:

- El número de máquinas sobre las que se distribuye el sistema (una o dos).
- La presencia o la ausencia Blockchain.
- El número de validadores de la red Besu.

La siguiente tabla, 6.3, resume las cinco configuraciones diseñadas:

ID de experimento	Máquinas	Blockchain	Validadores
E1	1	Sí	4
E2	1	No	—
E3	2	No	—
E4	2	Sí	4
E5	2	Sí	7

Tabla 6.3: Configuraciones experimentales

En todas las configuraciones, el entrenamiento federado se ejecuta con dos clientes (los *members* 2 y 3) durante 30 rondas, con 3 épocas locales por ronda y los hiperparámetros descritos en el capítulo anterior. De este modo, la diferencia entre los experimentos radica en los tres factores mencionados, lo que permite usarlos de variables y hacer comparaciones independientes entre ellos.

6.2.1 Despliegue del entorno

En las configuraciones de **una máquina** (E1 y E2), todos los componentes (servidor de Flower, ambos clientes, red Besu e IPFS cuando aplica) se ejecutan sobre la misma máquina.

En las configuraciones de **dos máquinas** (E3, E4 y E5), el sistema se reparte de la siguiente manera: la máquina 1 aloja el servidor de Flower (y, en los experimentos con Blockchain, el nodo RPC principal de la red Besu y el nodo IPFS), mientras que la máquina 2 ejecuta los dos clientes y realiza su entrenamiento del modelo de IA. La comunicación entre ambas se realiza a través de la red local Wifi mediante las direcciones IP correspondientes. Cabe destacar que es importante que en los experimentos E3, E4 y E5 el reparto del sistema se

mantenga igual y que las diferencias sean que exista o no la presencia de la Blockchain y la variación del número de validadores. En el caso de que se hubiera cambiado la distribución entre los componentes del sistema los resultados no hubieran sido precisos y válidos desde un punto de vista de una comparativa justa.

6.2.2 Comparaciones planteadas

El diseño de las cinco configuraciones permite realizar comparaciones en las que solo varía una de las tres variables, lo que permite una comparación aislada de los diferentes factores que se han definido. Las comparaciones son las siguientes,

- **Coste de la Blockchain**: Se evalúa comparando las mismas configuraciones que solo difieren en la presencia o la ausencia de la red Blockchain. En local se compara E1 (con Blockchain) frente a E2 (sin) y en distribuido, E4 (con Blockchain y 4 validadores) frente a E3 (sin). En este punto se elige la red de Blockchain con 4 validadores en vez de la de 7, ya que ésta es suficiente y permite ver la sobrecarga que hay al levantar la red, que es el objetivo de este experimento. Posteriormente se compararán los resultados respecto a la red más grande, de 7 validadores.

- **Coste de la distribución**: Se evalúa comparando configuraciones que solo difieren en el número de máquinas. Se hará por tanto una evaluación de los resultados sin Blockchain (E2, local) frente a E3 (distribuido) y otra evaluación con Blockchain y 4 validadores (E1, local) frente a E4 (distribuido).

- **Efecto del número de validadores**: se evalúa comparando E4 (4 validadores) frente a E5 (7 validadores), ambas en distribuido y con Blockchain.

Se ha definido este conjunto específico de comparaciones para evitar evaluar diferentes variables de manera simultánea y generar así cruces, cambiando varios factores a la vez. Por ejemplo un escenario que no se ha contemplado por estas razones podría ser la comparación del E2 y el E4, donde varía la distribución de las máquinas y la presencia de la Blockchain, lo cual podría no llegar a dar unos resultados justos y precisos para el ámbito de este trabajo.

6.2.3 Aspectos adicionales de la ejecución

Cada configuración se ha ejecutado una vez, completando las 30 rondas de entrenamiento definidas, por lo que los datos de las métricas para cada configuración se corresponden a una ejecución determinada. Ya que el proceso de entrenamiento es determinista en su estructura (mismo modelo, mismos hiperparámetros, misma partición de datos) y que las métricas se recogen continuamente a lo largo de 30 rondas, los valores medidos en cada ronda ofrecen una estimación representativa del comportamiento del sistema en cada configuración. No obstante, al tratarse de una única ejecución por configuración, las medidas de tiempo pueden ser afectadas por diferentes situaciones de variabilidad del entorno (virtualización de WSL, latencia de la red, etc...), por lo que las pequeñas diferencias entre configuraciones deben interpretarse con cautela. La repetición de cada experimento en múltiples ejecuciones para obtener intervalos de confianza se plantea como línea de trabajo futuro.

6.3 Resultados

A continuación se presentan y se analizan los resultados obtenidos en las cinco configuraciones experimentales propuestas y descritas en el apartado anterior. El análisis se organiza en torno a cuatro verticales: la convergencia del modelo, el coste temporal de integrar la Blockchain, el efecto de distribuir el sistema en varias máquinas y el impacto del número de validadores. Además, se analizará el consumo de gas de las operaciones sobre la red de Blockchain (on-chain). La métrica de referencia para el tiempo por ronda es la **mediana** (salvo que se indique lo contrario), debido a su robustez frente a valores atípicos (outliers) en el contexto de una única ejecución por configuración.

6.3.1 Convergencia del modelo

La figura 6.1 muestra la evolución de la función de pérdida (*cross-entropy*) sobre el conjunto de validación a lo largo de las 30 rondas. Se observa que la función de pérdida disminuye rápido durante las primeras rondas, alcanzando un mínimo en torno a la cuarta ronda ($\approx 0{,}24$), para después crecer de forma sostenida hasta valores cercanos a 0,90 al final del entrenamiento.

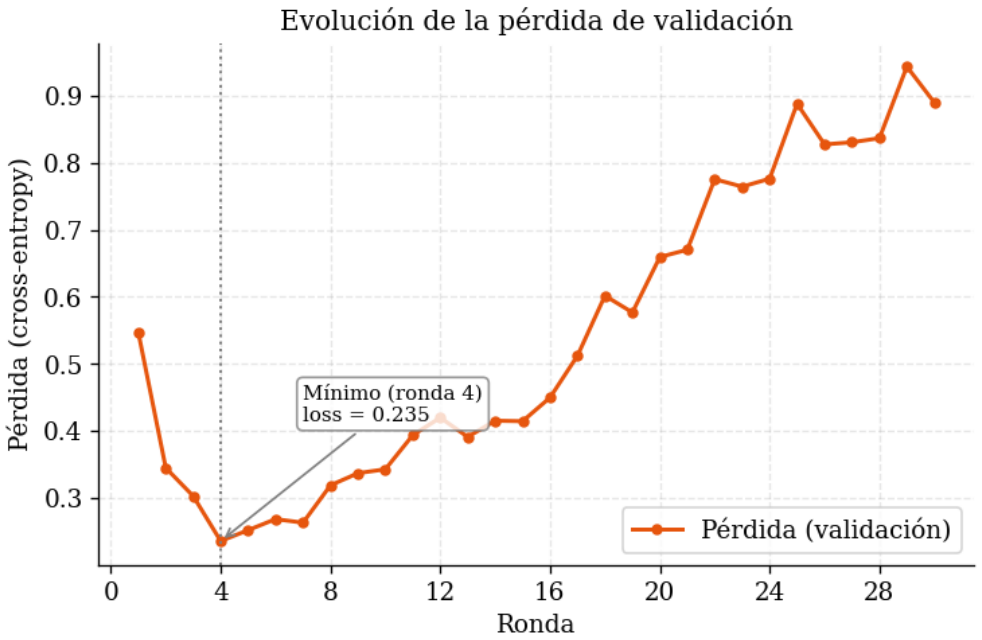


Figura 6.1: Evolución de la pérdida de validación a lo largo de las 30 rondas.

Este comportamiento, aparentemente contradictorio con una accuracy que se mantiene estable en torno al 97%, es un caso clásico de **sobreajuste** (*overfitting*). El modelo continúa ajustándose a los datos de entrenamiento ronda tras ronda, lo que le lleva a realizar predicciones cada vez más extremas y sobreconfiadas. Como la accuracy únicamente contabiliza si la clase predicha es la correcta, no se ve afectada; sin embargo, la *cross-entropy* penaliza los errores cometidos con alta confianza, lo que explica que el crecimiento de la pérdida sea cada

vez mayor. Es decir, el modelo acierta con la misma frecuencia, pero sus errores son cada vez más "seguros".

La causa principal es la combinación de un número elevado de iteraciones totales (3 épocas locales × 30 rondas = 90 pasadas sobre los datos) frente a un dataset y una arquitectura relativamente reducidos. El mínimo de la ronda 4 sugiere que un mecanismo de *early stopping* habría detenido el entrenamiento en ese punto, obteniendo el mejor modelo posible y ahorrando además varias rondas de cómputo y de transacciones on-chain. No obstante, es necesario aclarar que el objetivo de este trabajo no es desarrollar un modelo lo más preciso posible, sino comprobar la viabilidad de implementar Blockchain en el proceso de entrenamiento de un modelo de IA y comprobar que los tiempos de ejecución obtenidos se pueden asumir de cara a un despliegue en un entorno real. Es por ello que se ha decidido mantener esta configuración del entrenamiento: para poder realizar las 30 rondas fijas, necesarias para obtener sus tiempos de ejecución y realizar así las comparaciones. En este contexto, el modelo de IA actúa como un elemento de referencia sobre el que medir el comportamiento del sistema, y no como el objeto central del experimento.

Conviene destacar también que este fenómeno es **independiente de la capa Blockchain**: la integración con Blockchain e IPFS no interviene en el proceso de aprendizaje, sino que únicamente registra y verifica los modelos, por lo que la convergencia es idéntica en todas las configuraciones. La reducción del sobreajuste (mediante *early stopping*, regularización, reducción del número de epochs, etc...) se plantea como línea de trabajo futuro.

6.3.2 Visión general de los tiempos

La Tabla 6.4 resume el tiempo total por ronda en las cinco configuraciones, y la Figura 6.2 muestra su evolución a lo largo de las 30 rondas.

Exp.	Máq.	Config.	Mediana (s)	Media (s)	Desv. (s)
E1	1	4 val · BC	89,37	91,84	6,34
E2	1	sin BC	84,15	82,18	6,95
E3	2	sin BC	64,26	64,27	0,74
E4	2	4 val · BC	71,29	77,05	12,63
E5	2	7 val · BC	70,88	73,40	4,71

Tabla 6.4: Tiempo total por ronda en cada configuración.

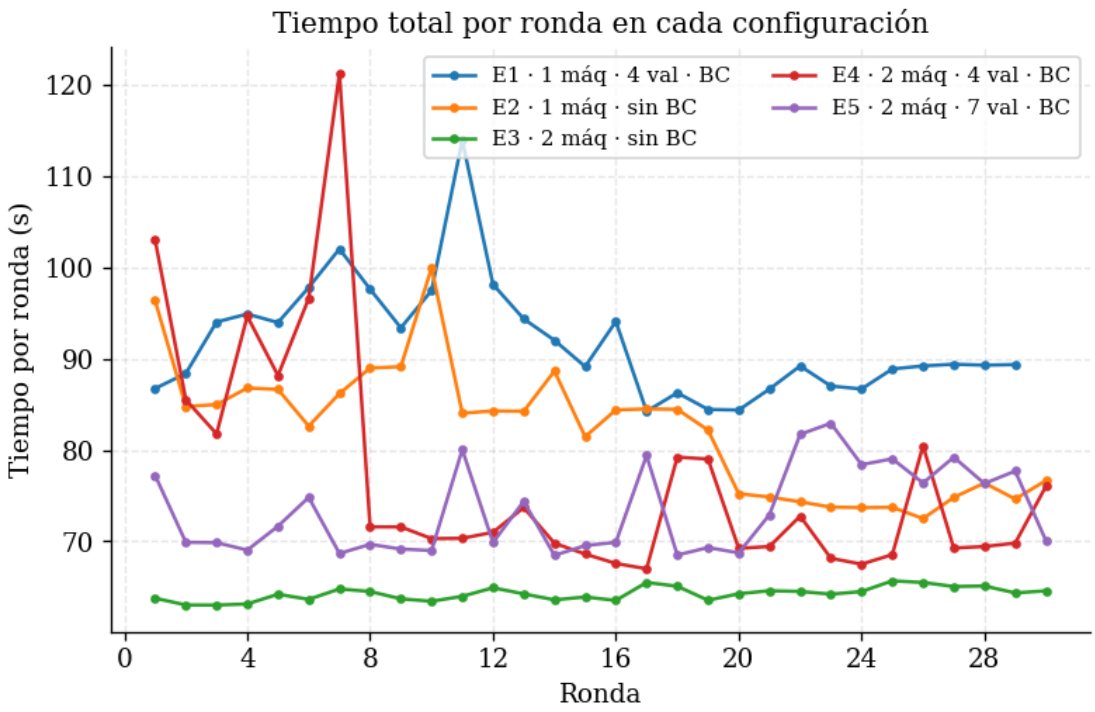


Figura 6.2: Evolución del tiempo total por ronda en las cinco configuraciones.

Antes de analizar las comparaciones, conviene señalar la elevada desviación típica de E4 (12,63 s) frente al resto de configuraciones distribuidas (0,74 y 4,71 s). Esta dispersión se debe a un valor atípico que se trata a continuación.

Tratamiento de valores atípicos

Durante el experimento E4 se registró en la ronda 7 un tiempo total de 121,3 s, muy superior al del resto de rondas de la misma configuración. Este pico se produce por una bajada puntual de los recursos disponibles en el entorno de ejecución, provocada por la capa de virtualización de WSL y por la limitada memoria disponible (7,6 GB). Un análisis mediante el criterio del rango intercuartílico identifica esta ronda como observación atípica. Dado que este valor no es representativo del comportamiento estable del sistema y que cada configuración se ejecutó una única vez, se reporta la **mediana** como estadístico principal por su robustez frente a valores extremos, indicando también la media y la desviación típica para mayor transparencia. Conviene destacar que el entorno de desarrollo utilizado para la experimentación es propenso a la aparición de este tipo de fluctuaciones de valores, debido a la sobrecarga a la que se encuentran sometidas ambas máquinas y con los pocos recursos disponibles. En un entorno con mayor capacidad de cómputo y recursos se esperarían tiempos menores y una menor dispersión entre rondas, si bien las tendencias observadas y explicadas a continuación entre las configuraciones se mantendrían.

6.3.3 Coste de la integración con Blockchain

En este apartado se expondrán los costes comprobados de la presencia de Blockchain en el sistema frente a un entrenamiento distribuido tradicional. Para aislar el coste de la capa de trazabilidad se comparan configuraciones idénticas que solo difieren en la presencia de la Blockchain: E1 frente a E2 en local, y E4 frente a E3 en distribuido. La Figura 6.3 muestra esta comparación.

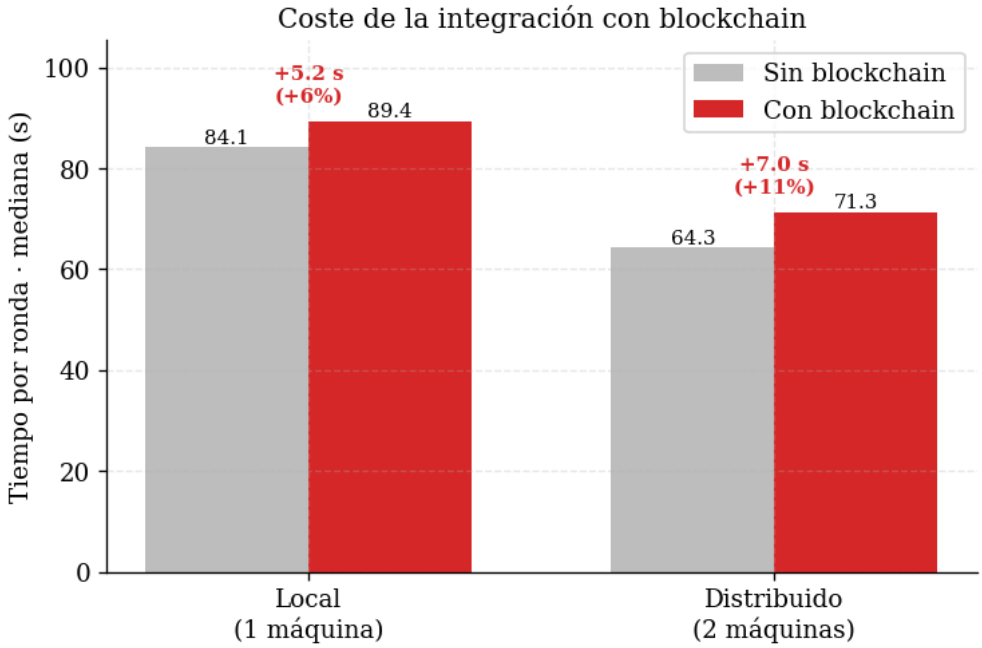


Figura 6.3: Coste temporal de la integración con Blockchain, en local y en distribuido.

En el escenario local, la introducción de Blockchain eleva el tiempo mediano por ronda de 84,1 s a 89,4 s, lo que supone un sobrecoste de 5,3 s (+6%). En el escenario distribuido, el tiempo pasa de 64,3 s a 71,3 s, un sobrecoste de 7,0 s (+11%). En ambos casos el sobrecoste es moderado y se corresponde con las operaciones añadidas en cada ronda: la subida del modelo a IPFS, el registro del CID en la Blockchain, la espera a la confirmación de la transacción y las verificaciones cruzadas entre cliente y servidor.

Estos resultados permiten afirmar que el precio de dotar al sistema de trazabilidad, integridad y no-repudio es asumible: por un incremento de entre el 6% y el 11% en el tiempo por ronda, el sistema gana garantías de auditoría de las que un sistema de aprendizaje federado clásico carece por completo.

6.3.4 Efecto de la distribución en varias máquinas

El segundo eje de análisis es el efecto de distribuir el sistema en dos máquinas frente a ejecutarlo en una sola. La comparación se realiza tanto sin Blockchain (E2 frente a E3) como con Blockchain (E1 frente a E4), como muestra la Figura 6.4.

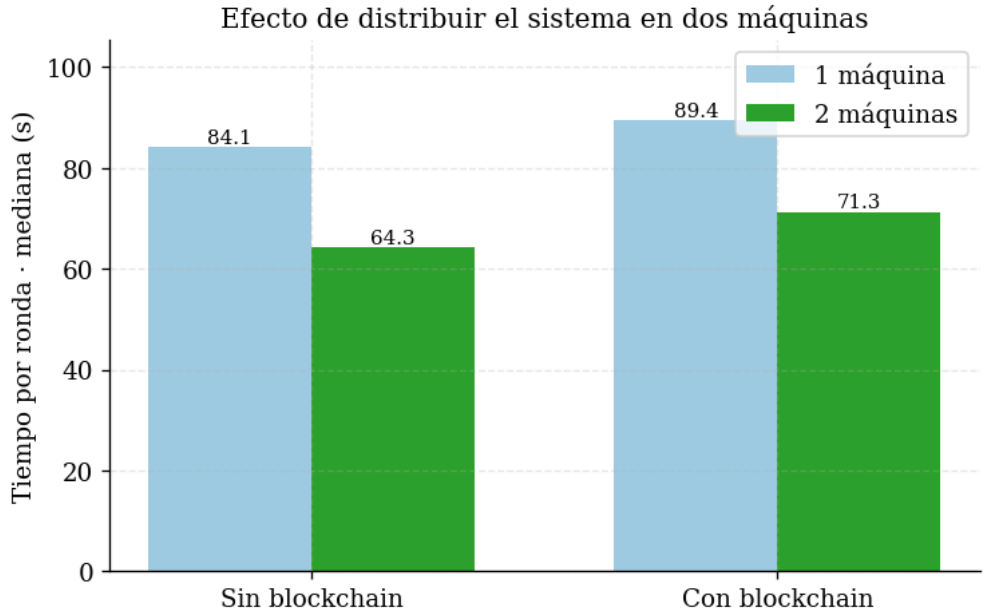


Figura 6.4: Efecto de distribuir el sistema en dos máquinas.

Distribuir el sistema en dos máquinas **reduce** el tiempo por ronda en lugar de aumentarlo: sin Blockchain, el tiempo mediano baja de 84,1 s a 64,3 s; con Blockchain, de 89,4 s a 71,3 s. La explicación está en el reparto de la carga: en la configuración de una máquina, todos los componentes (servidor, ambos clientes, red Besu e IPFS) conviven en los mismos 8 hilos y los 7,6 GB de memoria disponibles, lo que genera una contención de recursos importante y notable. Al separar el servidor (Máquina 1) de los clientes (Máquina 2), cada componente dispone de más recursos y el conjunto se ejecuta con mayor fluidez, compensando con creces la latencia de red introducida por la comunicación entre máquinas.

Este resultado es relevante porque sugiere que el cuello de botella en el escenario local no es el algoritmo ni la Blockchain, sino la limitación de recursos de la máquina, y que el sistema escala adecuadamente al distribuirse.

6.3.5 Impacto del número de validadores

El tercer eje analiza el efecto de aumentar el número de validadores de la red Besu de 4 (E4) a 7 (E5). En cuanto al tiempo total por ronda, ambas configuraciones presentan valores medianos prácticamente idénticos (71,3 s y 70,9 s), lo que indica que el número de validadores apenas influye en el tiempo global. Esto es coherente con el hecho de que el tiempo de ronda está dominado por el entrenamiento local del modelo, no por el proceso de consenso.

Para apreciar el efecto real del consenso es necesario observar el **tiempo de confirmación de transacción**, una métrica que aísla el coste de la Blockchain del resto del proceso. La figura 6.5 muestra este tiempo para las transacciones del cliente (`registerClientModel`) y del servidor (`updateModel`) en las tres configuraciones con Blockchain.

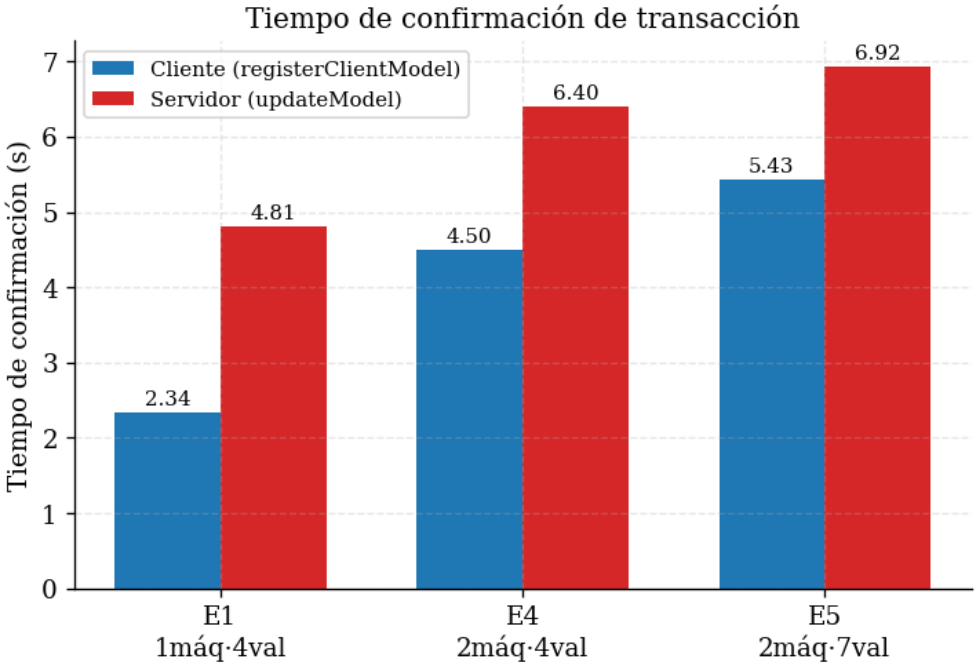


Figura 6.5: Tiempo medio de confirmación de transacción en las configuraciones con Blockchain.

Aquí sí se aprecia la tendencia que se esperaba: el tiempo de confirmación crece al aumentar el número de validadores. En las transacciones del cliente, pasa de 4,50 s con 4 validadores (E4) a 5,43 s con 7 (E5); en las del servidor, de 6,40 s a 6,92 s. Este incremento se debe al algoritmo de consenso IBFT 2.0, que con más validadores, cada bloque requiere la firma de un mayor número de nodos antes de considerarse confirmado, lo que añade latencia. Se observa además que la configuración local (E1) presenta los tiempos de confirmación más bajos (2,34 s en el cliente), al no existir latencia de red entre los nodos. La conclusión es que aumentar el número de validadores refuerza la robustez y la tolerancia a fallos de la red (con 7 validadores se toleran hasta 2 nodos defectuosos frente a 1 con 4 validadores), a costa de un incremento moderado en la latencia de confirmación que, sin embargo, queda absorbido dentro del tiempo total de ronda sin afectarlo de forma apreciable.

6.3.6 Consumo de gas

Por último, se analiza el consumo de gas de las operaciones on-chain. La Figura 6.6 muestra el gas consumido por las transacciones del cliente y del servidor a lo largo de las rondas.

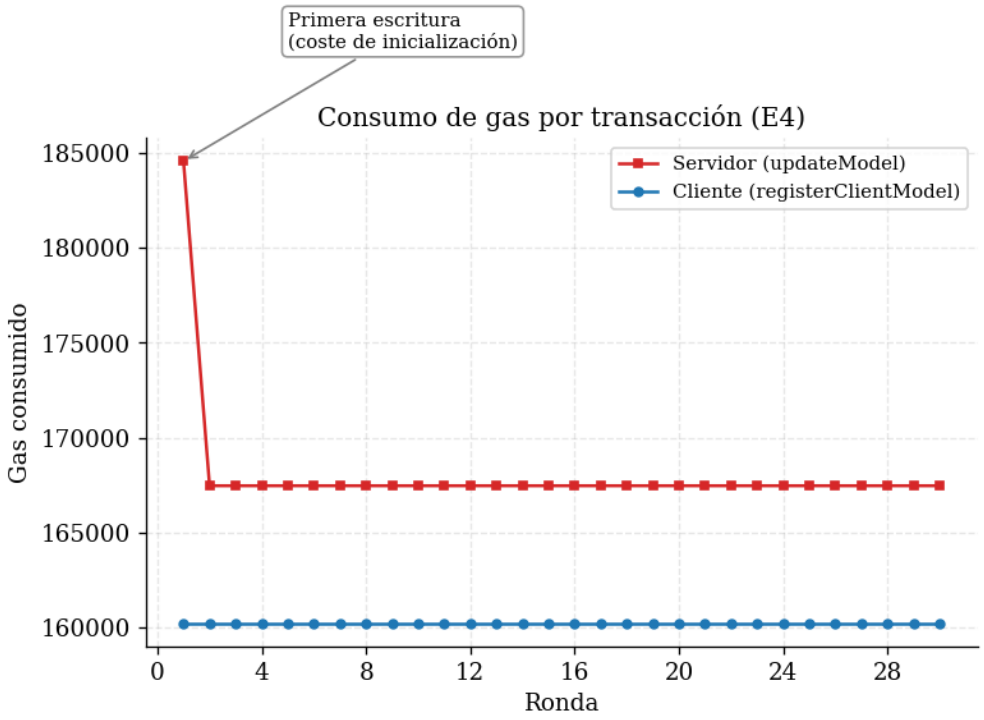


Figura 6.6: Consumo de gas por transacción a lo largo de las rondas.

El consumo de gas es **constante** a lo largo de todas las rondas: aproximadamente 160.200 unidades para la transacción del cliente (`registerClientModel`) y 167.460 para la del servidor (`updateModel`). Que estas operaciones mantengan un coste constante es coherente con el diseño del contrato: ambas operaciones almacenan un CID (una cadena de longitud fija) y unos pocos campos adicionales, por lo que su coste no depende ni de la ronda ni del contenido del modelo. La única excepción es la primera transacción del servidor, que consume 184.560 unidades frente a las 167.460 del resto. Este incremento se debe a que la primera escritura en una posición de almacenamiento del contrato (inicializar un valor desde cero, así como activar el *flag* `hasModel`) tiene un coste de gas superior al de las actualizaciones posteriores. Es un comportamiento básico y común de la máquina virtual de Ethereum.

Dado que la red es permisionada y el *gas price* está configurado a cero, este consumo no representa un coste económico, pero sí constituye una métrica objetiva del trabajo computacional que cada operación impone a la red, y el hecho de que el coste sea constante confirma que el comportamiento del contrato es predecible y que el coste de registro por ronda no crece con el número de rondas.

6.3.7 Discusión global

Los resultados obtenidos permiten responder a las preguntas planteadas al inicio del proyecto. En primer lugar, el coste de integrar una capa de trazabilidad basada en Blockchain es **moderado y asumible**: entre un 6% y un 11% de incremento en el tiempo por ronda, a cambio de garantías de integridad, trazabilidad y no-repudio. En segundo lugar, el sistema **escala adecuadamente** al distribuirse ya que la distribución en dos máquinas mejora el rendimiento al aliviar la contención de recursos. En tercer lugar, el número de validadores introduce un **compromiso entre robustez y latencia**: más validadores aumentan la tolerancia a fallos a costa de una mayor latencia de confirmación, que no obstante queda absorbida en el tiempo total de ronda. Finalmente, el consumo de gas es **constante y predecible**, con lo que se puede afirmar que el diseño del contrato es correcto en este aspecto.

En conjunto, estos resultados demuestran la viabilidad de integrar Blockchain en un sistema de aprendizaje federado para dotarlo de trazabilidad, con un sobrecoste contenido y un comportamiento que es predecible.

7 Conclusiones

En este último capítulo se recapitula el trabajo realizado, se comprueba el cumplimiento de los objetivos planteados al inicio del proyecto, se exponen las principales conclusiones extraídas y se proponen las líneas de trabajo futuro que quedan abiertas.

7.1 Recapitulación del trabajo

A lo largo de este Trabajo de Fin de Grado se ha diseñado, implementado y evaluado un sistema que integra la tecnología Blockchain en el proceso de entrenamiento de un modelo de Inteligencia Artificial distribuido. El sistema combina una red de aprendizaje federado, construida con el framework Flower, con una red de Blockchain permisionada basada en Hyperledger Besu, empleando IPFS como capa de almacenamiento distribuido de los modelos. El resultado es un sistema en el que cada contribución de cada participante queda registrada de forma inmutable y verificable, dotando al entrenamiento distribuido de una capa de trazabilidad, integridad y auditoría de la que un sistema federado tradicional no dispone.

7.2 Revisión de los objetivos

A continuación se revisa si se han cumplido los objetivos específicos planteados al inicio de esta memoria:

- **Estudio del estado del arte**: se ha llevado a cabo un estudio de los fundamentos del aprendizaje federado y de la tecnología Blockchain, así como de las investigaciones previas que motivan el proyecto, sentando las bases teóricas recogidas en el marco teórico. *Objetivo cumplido.*

- **Diseño de la arquitectura híbrida**: se ha definido una arquitectura que se compone de la red Blockchain y la red de entrenamiento federado, estableciendo qué información se registra on-chain y cómo interactúan ambos sistemas. *Objetivo cumplido.*

- **Despliegue de la red Blockchain y el desarrollo del smart contract**: se ha desplegado una red Hyperledger Besu permisionada y se ha desarrollado el contrato `FLRegistry`, el cual registra las contribuciones de los clientes y los modelos agregados de cada ronda. *Objetivo cumplido.*

- **Implementación del aprendizaje federado**: se ha desarrollado un sistema funcional con un servidor agregador y varios clientes que entrenan de forma colaborativa un modelo de clasificación de imágenes de tráfico sin centralizar sus datos. *Objetivo cumplido.*

- **Integración de ambos sistemas**: se han conectado ambas redes mediante IPFS y un mecanismo de verificación cruzada entre cliente y servidor que garantiza la integridad de los modelos y permite detectar manipulaciones producidas por ambas partes. *Objetivo cumplido.*
- **Evaluación experimental**: se ha evaluado el sistema en cinco configuraciones distintas, midiendo el coste adicional tanto de tiempo como de recursos de la capa Blockchain y su comportamiento al variar el número de máquinas y de validadores. *Objetivo cumplido.*
- **Respuesta al dilema entre seguridad y rendimiento**: los experimentos han permitido medir la latencia añadida por la Blockchain y valorar su asumibilidad, dando respuesta a la pregunta central del proyecto. *Objetivo cumplido.*

Viendo que todos los objetivos específicos se han cumplido, puede afirmarse por tanto que se ha alcanzado el objetivo general del trabajo, que es: validar la viabilidad de integrar Blockchain en un proceso de entrenamiento distribuido y determinar si su coste adicional es asumible.

7.3 Conclusiones principales

A partir de los resultados obtenidos se extraen las siguientes conclusiones:

- **La integración es viable y el sobrecoste es asumible**. La incorporación de la capa Blockchain introduce un incremento moderado en el tiempo por ronda, de entre el 6% y el 11% según la configuración. A cambio de ese coste, el sistema aumenta en garantías de trazabilidad e integridad, cosa que un sistema federado clásico no puede ofrecer. Este resultado responde afirmativamente a la pregunta planteada en la introducción sobre si el coste de la auditabilidad es aceptable.
- **La verificación cruzada protege a ambas partes**. El mecanismo diseñado en el que el cliente comprueba el modelo global contra la cadena antes de entrenar y el servidor verifica cada contribución antes de agregarla, permite detectar tanto un servidor que distribuya un modelo manipulado como un cliente que declare una contribución falsa. Esto aborda directamente la problemática del usuario malicioso planteada en la introducción.
- **El sistema escala adecuadamente al distribuirse**. En vez de penalizar el rendimiento, la distribución en dos máquinas lo mejora, al aliviar la contención de recursos que se produce al ejecutar todos los componentes en una única máquina.
- **Existe un compromiso entre robustez y latencia**. Aumentar el número de validadores incrementa la tolerancia a fallos de la red a costa de una mayor latencia de confirmación, si bien este incremento queda absorbido dentro del tiempo total de ronda sin afectarlo de forma apreciable en las experimentaciones realizadas.
- **El coste on-chain es constante y predecible**. El consumo de gas de las operaciones del contrato se mantiene estable a lo largo de todo el entrenamiento, lo que confirma la eficiencia y previsibilidad del diseño propuesto.

7.4 Limitaciones

El trabajo presenta algunas limitaciones que conviene reconocer. Los experimentos se han ejecutado sobre equipos del día a día (portátil y sobremesa) con recursos limitados y uno bajo virtualización WSL, lo que introduce variabilidad en las medidas absolutas de tiempo, aunque no afecta a la validez de las comparaciones. Asimismo, cada configuración se ha ejecutado una única vez, por lo que las conclusiones se apoyan en tendencias y no en intervalos de confianza estadísticos. Por último, el modelo de IA empleado, al no ser el foco del trabajo, presenta sobreajuste a partir de cierto número de rondas, fenómeno que no afecta a la evaluación del sistema pero que limita la calidad del modelo en sí.

7.5 Trabajo futuro

A partir del trabajo realizado se abren varias líneas de mejora y ampliación:

- **Mitigación del sobreajuste**, mediante la adición de *early stopping*, técnicas de regularización o el ajuste del número de épocas y rondas, de cara a obtener modelos de mayor calidad.
- **Evaluación en escenarios no-IID**, en los que los datos estén distribuidos de forma heterogénea entre los clientes, situación más realista en despliegues reales de aprendizaje federado.
- **Defensa frente a ataques de envenenamiento**, incorporando mecanismos que detecten contribuciones mal entrenadas aposta, más allá de la verificación de integridad actual. Al igual que la prevención de la introducción de usuarios maliciosos en la red mediante mecanismos de defensa complejos.
- **Mecanismos de reputación on-chain**, que aprovechen el registro inmutable de contribuciones para ponderar o filtrar la participación de los clientes en función de su comportamiento histórico.
- **Gestión robusta de claves**, sustituyendo el almacenamiento de claves privadas en el código por un sistema de gestión de secretos adecuado para un entorno de producción.
- **Validación estadística**, repitiendo cada configuración múltiples veces para obtener intervalos de confianza y reforzar la solidez de las conclusiones.
- **Despliegue en un entorno real**, llevando el sistema a un escenario de vehículos o dispositivos conectados que permita validar su comportamiento bajo condiciones de tiempo real y hardware limitado, tal como motiva el caso de uso planteado en la introducción.

Como reflexión final, este trabajo confirma que la integración de Blockchain en el entrenamiento distribuido de modelos de Inteligencia Artificial no solo es técnicamente posible, sino que su coste es asumible frente a las garantías de seguridad y auditoría que aporta, abriendo la puerta a sistemas de aprendizaje colaborativo más fiables y transparentes.

Bibliografía

Benet, J. (2014). Ipfs - content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*.

Beutel, D. J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., … Lane, N. D. (2020). Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*.

Buterin, V. (2014). *Ethereum: A next-generation smart contract and decentralized application platform.* `https://ethereum.org/en/whitepaper/`. (Accedido en 2026)

CloudLab-AIA. (2026). *FL-Blockchain: Federated Learning & Blockchain for Vehicular Event Recognition.* `https://github.com/cloudlab-aia/FL-Blockchain`. (Repositorio en GitHub. Accedido: 2026-06-15)

ConsenSys. (s.f.-a). *Goquorum documentation.* `https://consensys.io/blog/what-is-consensys-quorum`. (Accedido en 2026)

ConsenSys. (s.f.-b). *Quorum developer quickstart.* `https://consensys.io/developers/quickstart-and-tutorials`. (Accedido en 2026)

ConsenSys. (s.f.-c). *Tessera documentation.* `https://consensys.io/blog/tessera-the-privacy-manager-of-choice-for-consensys-quorum-networks`. (Accedido en 2026)

Ethereum Foundation. (s.f.-a). *Consensus mechanisms.* `https://ethereum.org/en/developers/docs/consensus-mechanisms/`. (Accedido en 2026)

Ethereum Foundation. (s.f.-b). *Gas and fees.* `https://ethereum.org/en/developers/docs/gas/`. (Accedido en 2026)

Ethereum Foundation. (s.f.-c). *Networking layer.* `https://ethereum.org/en/developers/docs/networking-layer/`. (Accedido en 2026)

Ethereum Foundation. (s.f.-d). *Nodes and clients.* `https://ethereum.org/en/developers/docs/nodes-and-clients/`. (Accedido en 2026)

Flower Labs. (s.f.). *Flower documentation.* `https://flower.ai/docs/`. (Accedido en 2026)

Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press.

Hyperledger Besu. (s.f.). *Hyperledger besu documentation.* `https://docs.besu-eth.org/private-networks`. (Accedido en 2026)

Lamport, L., Shostak, R., y Pease, M. (1982). The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, *4*(3), 382–401.

McMahan, B., Moore, E., Ramage, D., Hampson, S., y Arcas, B. A. y. (2017). Communication-efficient learning of deep networks from decentralized data. En *Proceedings of the 20th international conference on artificial intelligence and statistics (aistats)*.

Menezes, A. J., van Oorschot, P. C., y Vanstone, S. A. (1996). *Handbook of applied cryptography*. CRC Press.

Moniz, H. (2020). The istanbul bft consensus algorithm. *arXiv preprint arXiv:2002.03613*.

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system.* `https://bitcoin.org/bitcoin.pdf`. (Accedido en 2026)

National Institute of Standards and Technology. (2015). *FIPS PUB 180-4: Secure Hash Standard (SHS)* (Inf. Téc.). U.S. Department of Commerce.

Protocol Labs. (s.f.). *Ipfs documentation.* `https://docs.ipfs.tech/`. (Accedido en 2026)

Pujol, F. A., Mora, H., Ramírez, T., Rocamora, C., y Bedón, A. (2024). Blockchain-based framework for traffic event verification in smart vehicles. *IEEE Access*, *12*, 9251–9266. doi: 10.1109/ACCESS.2024.3352738

Solidity Authors. (s.f.). *Solidity documentation.* `https://docs.soliditylang.org/`. (Accedido en 2026)

Wood, G. (2014). *Ethereum: A secure decentralised generalised transaction ledger (yellow paper).* `https://ethereum.github.io/yellowpaper/paper.pdf`. (Accedido en 2026)

Zheng, Z., Xie, S., Dai, H.-N., Chen, X., y Wang, H. (2018). Blockchain challenges and opportunities: a survey. *International Journal of Web and Grid Services*, *14*(4), 352–375.